String Conundrum: An Interactive System for Teaching and Visualising Python String Concepts

Luvuyo Matomela
University of Cape Town
Department of Computer Science
Cape Town, South Africa
MTMLUV001@myuct.ac.za

Abstract

Strings are fundamental to introductory programming, yet novice Python programmers often struggle with concepts such as indexing, slicing, immutability, comparison, and iteration because of incomplete or inaccurate mental models of program behaviour. **String Conundrum** is a web-based educational system designed to address these difficulties through structured lessons, string-focused visualisation, executable examples, adaptive practice, and misconception-targeted feedback.

This paper presents the author's backend contribution: sandboxed Python execution, an AST-based statement-level tracer that supplies the state and character indices required for visualisation, procedurally generated adaptive practice across eight string modules, persistent learner progress, and optional English–isiXhosa keyword and runtime-error support that leaves the underlying Python execution model unchanged.

The backend was evaluated against five measurable requirements (R1–R5) and through a three-cycle human evaluation with 35 students, supported by a cross-team peer review and a supervisor acceptance walkthrough. All 274 automated tests passed, a live PostgreSQL experiment verified the production persistence path, the sandbox contained all ten adversarial cases in its declared threat set, a 257-program tracer corpus matched observed Python behaviour throughout, round-trip transpilation preserved text and behaviour for all 18 challenge solutions, and the live endpoint stayed within its two-second target through 100 concurrent requests. Pooled System Usability Scale was 80.8/100 ($n = 33$) and did not degrade in the first-year cycle. An applied knowledge check found that 11 of 35 participants could not say what in-place character assignment does, 10 of them expecting no error at all: locally gathered evidence that the targeted misconception is real and persistent. Human and technical evidence converged on assessment robustness as the principal weakness, and a 1,600-question sweep run in response located a concrete grading-presentation defect. These results establish technical correctness, requirement satisfaction, and usability within the scope evaluated. They do not establish improved learning outcomes, for which a controlled pre-/post-test study remains necessary.

Keywords

computing education, novice programmers, Python strings, program visualisation, misconception-targeted feedback, adaptive practice, execution tracing, bilingual programming education

1 Introduction

Introductory programming requires learners to develop accurate mental models of program execution, not merely learn syntax. Novices commonly hold misconceptions about variables, assignment, control flow, scope, and data types [10,12], while Sorva's notional-machine framework [19] highlights the importance of understanding how execution changes program state. Such conceptual errors can be difficult to diagnose because they do not necessarily produce interpreter errors.

Python strings provide a useful setting for addressing this problem. Altadmri and Brown [1] found that string-comparison errors affected nearly 30,000 students in a large-scale programming dataset, while Izu and Weerasinghe [9] found that only 49% of students correctly completed a string-manipulation task after a semester of instruction. These findings motivate teaching approaches that make string behaviour visible rather than presenting only the final result.

String Conundrum is a web-based learning environment for Python strings that combines structured lessons, character-level visualisation, executable examples, adaptive practice, and misconception-targeted feedback. This paper focuses on the author's backend contribution: sandboxed execution of learner-written Python, AST-based statement-level tracing, procedurally generated adaptive practice across eight modules, authentication and persistent learner progress, and optional English–isiXhosa keyword and runtime-error support. The React frontend, misconception-feedback content, and translated lesson content were developed by the project partner.

The backend is evaluated against five principal requirements: **R1**, learner code executes independently of Flask and terminates within a fixed limit; **R2**, traces and resolved string indices agree with observed Python behaviour for supported constructs; **R3**, supported programs retain executable behaviour after English–isiXhosa round-trip transpilation; **R4**, execution remains below the project's two-second response-time target at expected deployment concurrency; and **R5**, procedurally generated practice has uniquely recoverable answers and accurately represents learner selections during grading and feedback.

These requirements connect the educational objective to directly measurable backend properties, and each is tied to a specific experiment in Section 5.

1.1 Aims and Scope

The project's central aim is to help novice programmers build more accurate mental models of Python strings by combining structured lessons, visible execution behaviour, executable examples, misconception-targeted feedback, and repeated adaptive practice in one string-focused environment. Within that wider two-person project, the aim of the work reported here is to design, implement,

and evaluate the backend capabilities that experience depends on: safe execution of learner-written Python, statement-level tracing that yields the character indices the visualisations consume, procedurally generated adaptive practice, and persistent learner progress and challenge data.

Two scope boundaries are stated explicitly because they bound every claim that follows. First, the contribution is the backend; the React frontend, misconception-feedback content, and translated lesson content are the project partner's work and are discussed only where they are needed to explain or evaluate the backend. Second, the evaluation establishes technical correctness, requirement satisfaction, and usability. It does not establish learning effectiveness, and no claim of improved learning outcomes is made anywhere in this paper.

2 Background Work and Research Gap

2.1 Background and Prior Work

Novice programming difficulties are linked more to inaccurate mental models of execution than to syntax. Pea [15] describes the "superbug" pattern, in which learners behave as though the computer interprets their intent; Sorva [19] argues that learners need an appropriate notional machine to understand how state changes; and Qian and Lehman [16] catalogue how such errors persist precisely because they need not produce interpreter errors. Strings are an acute case, since indexing, slicing, and immutability depend on behaviour that source code does not reveal, and the scale is documented: string-comparison errors affected nearly 30,000 students in a large-scale dataset [1], only 49% of students completed a string-manipulation task correctly after a semester [9], formatting is a specific hazard [8], and both sit inside the broader difficulty of learning to program [13,17]. Strings nonetheless remain underexplored as a connected learning problem rather than as scattered exercises.

Three research threads supply partial solutions, and the gap lies between them. *Program visualisation* exposes runtime state step by step: Online Python Tutor [5], Jeliot 3 [14], and the systems reviewed by Sorva et al. [20] all make execution visible, but being deliberately general-purpose they show that a variable changed without tying that change to a string-specific concept or to practice on it. *Adaptive tutoring* demonstrates the value of fitting activities to performance through knowledge tracing [4] and intelligent tutoring systems [21]; SMoL Tutor [11] comes closest by pairing misconception-aware quizzes with notional-machine visualisation, though for general language semantics, and interactive textbooks [12] and program-synthesis tutors [10] likewise adapt without a string-specific content model. *Localisation* is addressed by Hedy [7], which lowers the language barrier by translating keywords, but for syntax rather than for the semantics of any particular data type.

String Conundrum combines all three. It specialises the execution trace for string operations so the character indices driving the visualisation come from the same mechanism that reports state; it adapts practice using a string-specific misconception model spanning indexing, slicing, methods, immutability, comparison, and formatting; and it keeps standard Python semantics while offering English–isiXhosa keyword support as an optional accessibility layer rather than a separate language. Supplement S2 compares the systems feature by feature.

2.2 Ethical and Professional Considerations

Ethical clearance from the Faculty of Science Research Ethics Committee was obtained before recruitment, and every participant was briefed on the study and signed an informed consent form; participation was voluntary, with withdrawal possible at any stage without consequence. The application stores no personally identifying information: progress and answer data were anonymised, reported using anonymous participant identifiers, and used solely to evaluate the system. Because the system executes learner-submitted code on a university host, the containment boundary of Section 4.1 is a professional obligation as well as a requirement, and its limits are stated rather than implied.

3 Requirement Analysis and Design

3.1 Requirements and Their Evolution

Requirements were refined iteratively against observed system behaviour rather than treated as fixed descriptions of the proposal. Three revisions were significant enough to change the design. PostgreSQL replaced the originally proposed Firestore store once it was clear that progress, challenge completion, and leaderboard calculation are relational problems requiring aggregation and uniqueness constraints. Adversarial testing showed that a subprocess with a timeout alone did not restrict filesystem, network, process, or memory access, so the execution requirement was strengthened to demand containment of an explicit threat model. Adaptive practice grew from a Slicing-only prototype into a shared engine across all eight modules, separating common selection, grading, difficulty, and token handling from module-specific generation rules.

Six functional requirements followed. FR-1 to FR-6 require the backend to authenticate learners, safely execute submitted Python, trace it at statement level, support optional English–isiXhosa keywords, generate and grade adaptive practice across all eight modules, and persist progress, challenge outcomes, and leaderboard state. Six non-functional requirements cover performance (NFR-1), containment of untrusted code (NFR-2), correctness verifiable against an independent source of ground truth (NFR-3), predictable degradation (NFR-4), extensibility to new modules and keywords (NFR-5), and maintainability through separated, testable components (NFR-6). Expanded statements for both sets are in Supplement S3. The five requirements R1–R5 of Section 1 are the measurable subset used for evaluation: they restate FR-2, FR-3, FR-4, FR-5, NFR-1, NFR-2, and NFR-3 in a form that an experiment can falsify.

3.2 System Architecture

String Conundrum follows the three-tier architecture of Figure 1. The React frontend, the project partner's work, provides lessons, visualisations, Code Space, practice, and progress views. It communicates with a Flask REST API that coordinates authentication, execution, tracing, practice generation, grading, and progress. PostgreSQL stores persistent learner and challenge data; Firebase provides identity.

The division of responsibility is the load-bearing decision. The frontend owns presentation and interaction only, while the backend owns execution, tracing, grading, generated practice, and persistent state. Within the backend, API routing is separated from the

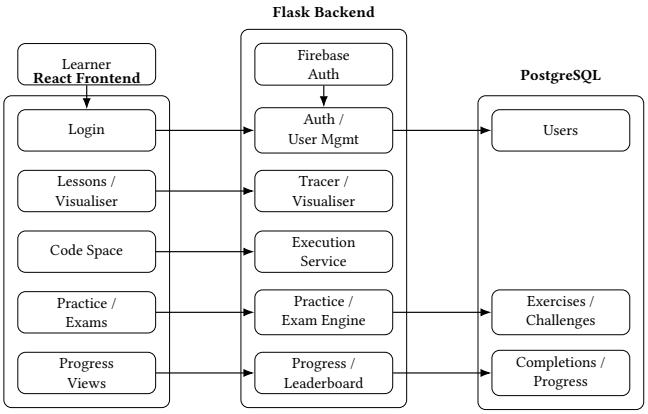


Figure 1: Realised three-tier architecture of String Conundrum, showing the React learning interface, Flask backend services, authentication, and persistent data layer. Components owned by this contribution are the Flask services and the persistence tier; the database schema is expanded in Supplement S18.

execution, tracing, practice, translation, and persistence services. This keeps educational logic in one place, and it is what makes the component-level experiments of Section 5 possible: each service can be exercised directly, against independent ground truth, without driving the interface.

3.3 Design Decisions and Trade-offs

Five decisions shaped the backend, and each traded something away. Table 1 sets out the alternatives considered, the criteria applied, and the cost of the option chosen. Three of the trade-offs deserve emphasis because they bound what the evaluation can subsequently claim.

Choosing an AST tracer over `sys.settrace` or debugger hooks was driven by what the visualisation needs: not line numbers and variable values, which a generic debugger supplies, but the specific character positions involved in an index or a slice. Interpreting the AST directly makes those positions available as first-class trace data. The cost is that the tracer implements a defined Python subset, so its correctness must be established against real Python execution for that subset rather than assumed (R2).

Choosing a restricted subprocess over in-process execution or a container was a deliberate acceptance of a weaker boundary. It is stronger than in-process execution, since a learner's infinite loop or crash cannot block Flask, and it was deliverable within the project. It is weaker than a hardened container, so security claims are confined to an explicit threat model (R1) and are not presented as general safety.

Choosing client-side keyword transpilation over a backend translation layer or a separate isiXhosa language keeps exactly one execution language in the system: the sandbox and tracer only ever see standard Python. This preserves Python semantics exactly and avoids a second correctness surface, at the price of limiting localisation to the supported vocabulary (R3).

Table 1: Decision matrix for the principal backend choices. The chosen option is bold; the criteria that decided each choice are in italics.

Decision	Alternatives (chosen in bold)	Trade-off accepted
Execution tracing	**AST tracer**; `sys.settrace`; debugger hooks. *Pedagogical detail, correctness, controllability*	Defined subset rather than full-language coverage, in exchange for exposing string indices and state explicitly
Persistence	**PostgreSQL**; Firestore; JSON file only. *Relational queries, constraints, deployability*	More schema work than a document store, in exchange for a direct fit to progress and leaderboard aggregation
Practice architecture	**Shared core plus module generators**; one engine per module; static bank only. *Extensibility, duplication, adaptive selection*	Requires a common abstraction, in exchange for avoiding eight divergent implementations
Bilingual support	**Client-side keyword transpilation**; backend translation; separate isiXhosa language. *Semantic fidelity, deployment complexity*	Localisation limited to the supported vocabulary, in exchange for one execution language and no second correctness surface
Isolation boundary	**Restricted subprocess**; in-process execution; container sandbox. *Safety, response time, implementation cost*	Weaker than container isolation, in exchange for being deliverable and stronger than in-process execution

4 System Development and Implementation

This section describes the implementation of the principal backend components introduced in Section 3, focusing on the mechanisms that support safe execution, string visualisation, adaptive practice, bilingual assistance, and persistent learner progress.

4.1 Authentication and Execution Sandbox

Protected routes verify a Firebase ID token and upsert the corresponding learner into PostgreSQL, so progress survives across sessions. Guest and display-name-only token paths were added to remove sign-up friction during usability testing while still giving each participant a stable backend identity.

Learner-submitted Python runs in a separate subprocess, for free exploration and for challenge grading alike. Three restrictions apply: a five-second wall-clock timeout, a restricted builtins dictionary that removes imports, file access, dynamic execution, and the common introspection routes back to them, and, on POSIX hosts, an `RLIMIT_AS` address-space cap against runaway allocation. The subprocess boundary is what keeps a learner's infinite loop or crash from blocking Flask. It is explicitly not container-grade isolation, so Section 5 evaluates it against a declared threat model instead of asserting general safety.

4.2 Statement-Level Execution Tracer

The tracer parses the program to an AST once, then walks the supported statements while maintaining an explicit environment dictionary representing current state. Each emitted step is a JSON record carrying the source line, event type, current variables, highlighted string indices, and a short learner-facing description; the ordered list of these records is what the frontend renders. Resolving indices inside the tracer rather than in the frontend is why one mechanism guarantees that the highlighted characters and the reported state cannot disagree.

Supported forms are assignments, augmented assignments, loops, conditionals, `print`, common string methods, indexing, and slicing. Negative and out-of-range slice bounds are normalised to Python's own behaviour, and a 200-step ceiling bounds trace size. Because this is a defined subset rather than the whole language, correctness is not assumed: R2 checks it differentially against real Python execution. The full AST contract, which also names what is deliberately out of scope, is in Supplement S4, with the tracer's operation and internal call sequence in Supplement S18.

4.3 English–isiXhosa Transpiler

Bilingual support is two separate mechanisms. The keyword transpiler applies an ordered mapping in the browser before any code reaches the backend, so the sandbox and tracer only ever receive standard Python. Ordering matters: multi-word mappings are substituted before their component words to prevent partial replacement, and substitution is suppressed inside comments and string literals. One `KEYWORD_PAIRS` table drives both the transpiler and the learner-facing keyword reference, so the two cannot drift apart (Supplements S5 and S18).

Separately, fifteen pre-translated templates give isiXhosa explanations for the principal exception categories the execution service produces, with detail such as the learner's line number inserted at request time; unrecognised errors fall back to the standard English path rather than altering execution, which is the NFR-4 degradation behaviour. This second mechanism translates the *explanation* of a failure, not the program, and so leaves execution and tracing untouched.

4.4 Adaptive Practice Engine

The practice engine serves all eight modules: Intro, Indexing, Slicing, Looping, Comparison, Methods, Immutability, and Formatting. Rather than drawing from a finite bank, each module composes a question from concept-specific templates crossed with varying strings, question kinds, answer formats, and difficulty tiers, which is what lets practice repeat without repeating itself.

The shared core owns tier progression, signed-token issue and verification, adaptive selection, and answer normalisation, while each module engine supplies only its own templates and grading rules behind a common `generate()` and `grade()` interface. This is NFR-5 extensibility in concrete form: a new module implements templates, not infrastructure. Adaptive selection raises the weight of templates tied to concepts a learner has recently missed while leaving other concepts reachable, so practice is targeted without trapping the learner in one area.

Two details carry most of the educational weight. Multiple-choice distractors are tied to named misconceptions wherever possible, so an incorrect selection identifies a likely misunderstanding that the wider system's feedback can address. And the signed token carries enough information to reconstruct the question at grading time, letting the backend recompute the expected answer without per-question server sessions, at the cost of having to reject tampered tokens explicitly, which R5 tests. Supplements S6 and S18 give the core/module boundary and the generation loop.

4.5 Data Model: Challenges, Progress, and Leaderboard

PostgreSQL stores learner identities, exercises, challenges, completion records, and module progress (schema in Supplement S18). Three properties of the model matter more than its shape. Completions carry uniqueness constraints, so a repeated correct submission cannot award a duplicate reward. Module-section completion is stored separately from question correctness, letting the system distinguish progressing through material from passing assessment on it. And leaderboard standing is derived from the persisted rows at read time rather than held as a separate mutable score, which removes the class of bug in which progress and leaderboard disagree. Challenges hold a canonical solution plus test cases; a submission runs against every stored case through the restricted grading path and counts as complete only if all pass.

The Flask API is the single boundary between interface and backend, keeping all execution, grading, and persistence logic out of the frontend. Practice endpoints share one route structure across all eight modules, so the frontend uses an identical workflow whichever module a learner studies. The endpoint inventory is in Supplement S7.

5 Evaluation Methodology

The evaluation combined automated regression testing, five requirement-specific technical experiments, and formative human evaluation. The technical experiments assess whether the implemented backend satisfies R1–R5 from Section 1, while the human evaluation examines the usability of the wider learning environment and identifies issues not readily captured by automated tests. The corresponding results are reported in Section 6.

5.1 Technical Evaluation

Every technical experiment pairs positive-path checks, which confirm that the intended learner workflow succeeds on normal input, with negative and boundary checks on malformed payloads, tampered tokens, unsupported constructs, and extreme resource or indexing conditions. The reason for insisting on both is that "no crash" is not evidence of correctness for any of R1–R5: a tracer that silently reports the wrong index, or a grader that accepts a forged token, fails without failing loudly. Table 2 states, for each requirement, the method, the workload, the success criterion declared before measurement, and the independent source of ground truth. Four methodological choices in that table need justification.

Ground truth is external wherever it can be. For R2 the reference is actual Python execution of the same program, never

a hand-written expectation of what the tracer should emit; for R3 it is the challenge's own test cases; for R5 a recomputed answer key. This is NFR-3 as an evaluation rule, and it is what stops an experiment confirming the implementation against itself. The same reasoning separates the `pytest` suite from FR-6: the suite runs against the JSON-file fallback, which isolates backend logic from database availability but is therefore not an integration test, so FR-6 was checked on the production host with a transient user by recording each completion twice and then querying the progress and leaderboard helpers, exercising uniqueness constraints and read-side aggregation rather than assuming them.

The success criteria were declared before measurement, not after. R1's threat model fixed seven categories in advance (CPU exhaustion, memory exhaustion, filesystem access, outbound networking, process creation or termination, import and introspection routes that could bypass restricted builtins, and benign controls that must still run); the ten cases were then written to cover them and submitted through the same path as learner code, a case counting as contained only if the restricted operation was blocked or terminated with Flask still serving. Fixing the categories first is what stops the threat set being retrofitted to the result, and equally why the result cannot generalise beyond them. R4's criterion, p95 below two seconds at classroom-scale concurrency, was likewise set in advance; levels to 1,000 were included to locate saturation, not to redefine the requirement, and a replicated follow-up compared two, four, and eight workers so that worker-count effects could be separated from a single-run artefact. Module-level R5 procedure is in Supplement S11.

5.2 Human Evaluation

The human evaluation was organised as three sequential design cycles rather than as one summative study. Each cycle produced a defect and improvement list that was acted on before the next cycle ran, so the cycles are deliberately not interchangeable replications; they are successive tests of a changing artefact. Two expert reviews ran alongside the student cycles: a cross-team peer review by the developers of a comparable Python-learning tool, which preceded Cycle 1, and a final acceptance walkthrough with the project supervisor, which sat between Cycles 2 and 3. Table 3 sets out the full design.

Thirty-five students participated in total: 11 in Cycle 1, 11 in Cycle 2, and 13 in Cycle 3. Cycles 1 and 2 recruited second-year, third-year, and Honours computing students. These participants are more experienced than the system's intended audience, so their sessions are treated as expert-adjacent usability and design review rather than as evidence about learners. Cycle 3 is the target-population cycle. It ran in a University of Cape Town Computer Science laboratory venue immediately after a first-year practical session; 10 of its 13 respondents self-identified as current UCT first-year students, and all 13 reported prior exposure to introductory Python. The three respondents who did not self-identify as first-years are retained in the reported totals and separated where the distinction matters.

5.2.1 Instruments, scoring, and analysis. Three instruments were held constant across all three cycles so results are comparable: the ten-item System Usability Scale (SUS) [3] as the principal usability measure; a five-point per-concept confidence measure collected as paired "before" and "after" ratings; and an applied knowledge check with objectively markable answers on indexing, slicing, in-place character assignment, and case-sensitive comparison. A single item asked whether the isiXhosa content aided understanding, and free-text prompts collected qualitative feedback.

The instruments then changed deliberately. Cycles 1 and 2 took free-text knowledge answers so reasoning, not only final answers, could be inspected; Cycle 3 replaced these with fixed options whose distractors encode specific misconceptions, making distractor analysis possible, and added an item on whether a string method mutates its receiver. Cycle 3 further added a ten-item battery in which each statement corresponds to one design claim, so endorsement attaches to a component rather than to the system in general; a mental-demand rating following the NASA-TLX demand dimension [6]; and a fixed issue checklist built from the problems Cycles 1 and 2 had raised in free text, converting open-ended reports into a prevalence measure over the target population.

SUS used the standard formula, with scales missing any item excluded rather than imputed, which removes one respondent from each of Cycles 1 and 2 and leaves 33 complete scales, read against the published mean of 68, the adjective anchors of Bangor et al. [2], and the curved bands of Sauro and Lewis [18]. Cycle 3's knowledge check was auto-marked and then independently regraded from the raw response strings; the regrade reproduced the instrument's totals for all 13 responses, so those marks are verified rather than asserted. Cycles 1 and 2 were coded against a stated rubric, three-level for the immutability item because the informative distinction is which mental model an answer implies rather than merely whether it is right: *correct* if the assignment is said to fail or the characters to be unassignable, *silent no-op* if the string is unchanged but no error is claimed, and *silent mutation* if the assignment is said to succeed so that `"Python"` becomes `"Jython"`. The rubric and every coded verbatim answer are in Supplement S12.

Paired confidence ratings were tested with the two-sided Wilcoxon signed-rank test [22], evaluated exactly by enumerating all 2^n sign assignments of the non-zero difference ranks, so no distributional approximation is relied on at these sample sizes; effect size is the matched-pairs rank-biserial correlation. Because the samples are small and the concept-level tests are not independent, these values are reported as descriptive support for the direction of change, not as confirmatory tests. The analysis script and anonymised responses accompany the submission, so every reported figure is reproducible from the raw data.

5.2.2 Expert and supervisor review. Three developers of PyTutor, a Python-learning tool built by another team for a comparable problem, completed an unscored review with no rubric, asked for direct criticism as builders of a similar system rather than as learners. This group is valuable because it is adversarial by construction: reviewers who have implemented their own grading and visualisation logic are well placed to find weaknesses in someone else's. The supervisor separately conducted a final acceptance walkthrough of the deployed build against a 30-item checklist over seven categories, with a four-level verdict key (accepted, accepted with comment, requires a fix before submission, not applicable), three open questions, and a signed statement; this verifies implemented functionality

Table 2: Requirement-to-evidence traceability: method, workload, pre-declared success criterion, and independent ground truth for each requirement. Observed outcomes are reported separately in Table 4.

Req.	Method	Workload / dataset	Success criterion (pre-declared)	Ground truth / validation
R1	Adversarial execution through the production path	10 programs covering 7 declared threat categories, plus a benign control	Restricted action blocked or terminated, control still runs, Flask stays available	Direct observation of the deployed host; the POSIX memory case verified on Linux specifically
R2	Differential trace checking	25 hand-authored programs; 232 generated programs over indices, omitted bounds, steps, loops, conditionals	Trace events, final state, and resolved indices agree with observed Python for the supported subset	Actual CPython execution of the same program, not an authored expectation
R3	Round-trip transpilation plus behavioural equivalence	All 18 canonical challenge solutions; transpiler unit tests	Round-tripped source matches the original as text *and* in pass/fail behaviour	The challenge's own stored test cases, run against both versions
R4	Load test of the live endpoint over the real network	Concurrency 5–1,000; replicated 5/100/300 subset; 2/4/8 worker comparison	p95 below 2 s at classroom-scale concurrency	Measured p50/p95/p99, throughput and failure rate; replication to separate variability from artefact
R5	Engine generation and grading checks plus a cross-module sweep	Full module test suite; 1,600 generated questions, 200 per module across all eight engines	Answers uniquely recoverable, no duplicate options, tampered tokens rejected, selections rendered as chosen	Recomputed answer key per question; direct `generate()`/`grade()` calls; interface inspection

against the agreed scope and is not a usability or learning measure. Four postgraduate reviewers with Human-Computer Interaction experience separately walked through the interface during the formative period, focusing on navigation, the visualiser, Code Space, and the module and exam screens; no scored instrument was administered, so their comments are reported as qualitative expert review alongside the peer developers' (Supplement S13).

5.3 Evaluation Scope and Limitations

The evaluation establishes technical correctness, requirement satisfaction, and usability. Seven limitations bound what may be concluded, and they are stated here rather than at the point where each result might otherwise look stronger.

On the technical side, the combined 257-program tracer corpus exercises only the supported Python subset, not the whole language; the ten sandbox cases test a declared threat model and are not a general security guarantee; and the R5 sweep gives broad cross-module coverage but a per-module sample that remains bounded, so it should be read together with the module test suite.

Four limitations bound the human evaluation. First, all three samples are small and self-selected, so interval estimates are wide and no subgroup comparison is adequately powered; the statistics describe these participants rather than estimating population parameters. Second, the confidence measure is retrospective, with both ratings given after the session on one questionnaire, which exposes it to recall bias and demand characteristics; it is therefore reported as a perception measure and never as a proxy for learning. Third, there is no control condition and no delayed post-test, so within-session change cannot be attributed to the system rather than to time on task, re-exposure to the material, or the practical that preceded Cycle 3. Fourth, each cycle evaluated a repaired build, so the three cycles cannot be pooled into one before-and-after comparison of the artefact; the cross-cycle figures describe a moving target.

Two design decisions partly offset these limits: the knowledge check is objectively markable and substantively identical across all three cycles, so it does not depend on self-report, and Cycle 3 ran with the intended population in the intended setting, which is where usability claims most need to hold. The comparative pre-/post-test learning study with a control condition proposed for the wider project had not been completed at submission, so no claim about learning outcomes is made anywhere in this paper.

6 Results and Discussion

6.1 Technical Results

Table 4 reports the outcome of each experiment defined in Table 2, alongside what that outcome does not establish. All five requirements were met within their declared scope, and the automated suite passes in full at 274 of 274 tests, of which 254 exercise the practice engines. That figure is much larger than an earlier measurement of 81 because coverage grew as the practice core was generalised from Slicing alone to all eight modules; a file-by-file breakdown is in Supplement S8, and the R3 round-trip results are broken out in Supplement S9. Two results require more than a table entry.

6.1.1 Backend performance (R4). Response time against the live deployment is shown in Figure 2, with the full measurements in Supplement S10. The two-second target holds cleanly through 100 concurrent requests, where p95 is 1.12 s, and is clearly exceeded by 300, where p50 alone is 3.27 s. The informative observation is that throughput never leaves a narrow band of 85 to 97 requests per second at any concurrency from 5 to 1,000. That is the signature of a fixed-capacity resource, namely the four-worker Gunicorn configuration, rather than of an execution path that scales badly. The replicated benchmark confirms it: doubling from four workers to eight at 100 concurrent requests raised throughput from 29.6 to 57.8 requests per second and cut p95 from 3.95 s to 1.76 s,

Table 3: Design of the three-cycle human evaluation, in chronological order, with the two expert reviews that ran alongside it.

Activity	Participants	Setting	Instruments	Role in the evaluation
Peer review	3 developers of PyTutor (comparable tool)	Cross-team, unscored	Open review form: first impressions, what works, what they would question, anything broken, overall	Adversarial technical critique from reviewers who have built comparable grading and visualisation logic.
Cycle 1	11 students: 2nd year, 3rd year, Honours	Remote / self-paced	SUS; retrospective pre–post confidence (4 concepts); 4 free-text knowledge items; isiXhosa item; free text	Formative. First contact with a complete build; surfaces functional defects and navigation problems.
Cycle 2	11 students: 2nd year, 3rd year, Honours	Remote / self-paced	As Cycle 1, plus a looping concept and extended free-text prompts on explanation quality	Formative. Tests the Cycle 1 fixes and probes explanation wording, feedback clarity, and practice coverage.
Supervisor	Project supervisor	Live walkthrough of the production deployment	30-item feature checklist over 7 categories, 4-level verdict key, 3 open questions, signed acceptance	Acceptance against the agreed project scope.
Cycle 3	13 students; 10 self-identified UCT first-years	UCT Computer Science laboratory venue, directly after a first-year practical	SUS; retrospective pre–post confidence (5 concepts); 5 auto-marked knowledge items with misconception distractors; 10-item design-claim battery; mental-demand rating; fixed issue checklist; most-helpful-feature choice; free text	Summative for usability and target-population fit. The only cycle run with the intended novice audience, in the intended setting.

whereas halving to two workers changed little. The saturation is therefore a deployment-configuration limit with a known remedy, not a property of the sandbox.

Failures appear abruptly rather than gradually: 0% through concurrency 500, then 48% at 800 and 58% at 1,000, all as connection resets. Whether those resets originate in the deployment's connection handling or in a single client machine opening 800 to 1,000 simultaneous outbound connections over one network path was not determined, and it is left as an open question rather than resolved in the system's favour.

6.1.2 Practice engine correctness (R5). R5 makes two demands. Every generated question must have a uniquely recoverable answer, and the interface must never present a learner's own selection as something other than what they chose. The 1,600-question cross-module sweep satisfied the first: 100% recoverability, no duplicate options in any multiple-choice or select-all output, and 100% rejection of tampered tokens (Supplement S11).

The second demand exposed a real defect, and it is worth reporting in full because it is the point at which the human and technical evaluations meet. Select-all questions frequently have more than one correct option. Before the fix, every correct option received identical solid-green, checkmarked styling once a question was graded, whether or not the learner had chosen it, so a learner who correctly picked one of two right answers watched a second option turn green as though they had picked it too. The grading logic was correct throughout; this was a presentation defect in `ChoiceList`.

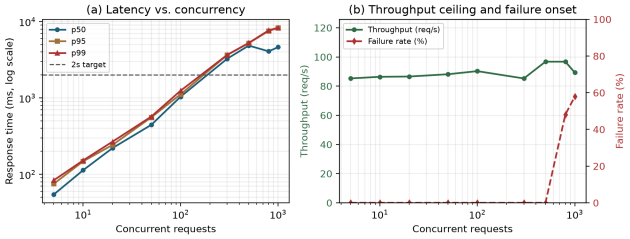


Figure 2: `POST /api/execute` against the live deployment. Part (a) shows p50, p95, and p99 response time against concurrency, on log-log axes, together with the two-second target. Part (b) shows that throughput holds a narrow band regardless of concurrency, while the failure rate stays at zero through 500 concurrent requests and then rises sharply.

The fix distinguishes four post-grading states, adding *correct-but-not-picked* as a dashed outline with no fill, so only what a learner actually clicked ever renders as selected. A duplicate-click report from Cycle 1 of the human evaluation, weeks earlier, is plausibly the same defect seen from the learner's side, though the two are not confirmed to be the same report.

6.2 Human Evaluation Results

6.2.1 Usability, perceived demand, and the design claims. Across the three cycles 35 students participated and 33 returned a complete SUS (Table 5a). The pooled mean is 80.8 out of 100 (95% CI 76.2 to 85.4),

Table 4: Observed outcomes for each requirement, with the limit on what each result establishes. Methods and predeclared criteria are in Table 2.

Item	Observed result	Does not establish
Suite	274/274 `pytest` tests pass (254 practice-engine, 20 API and translation)	Run on the JSON fallback, so not an integration test
FR-6	On the production database, duplicate submissions collapsed to exactly one section, one exercise and one challenge completion row, 5 stars total; progress and leaderboard helpers both returned the user	One transient user in one run
R1	All 10 adversarial cases contained; benign control still executed	Safety beyond the 7 declared threat categories
R2	25/25 hand-authored traces consistent with observed behaviour. Of 232 generated programs, 232 matched stdout, the 208 successful ones matched final state, and the 24 boundary/error cases matched both exception type and the point execution stopped	Correctness outside the supported AST subset
R3	10/10 transpiler unit tests; 18/18 exact-text round trips; 18/18 round-tripped solutions still pass their own tests with identical pass/fail behaviour	Fidelity beyond the supported vocabulary and corpus
R4	p95 1.12 s at concurrency 100, within target; p50 3.27 s at 300, outside it; throughput 85–97 req/s at every level; 0% failures to 500, 48% at 800, 58% at 1,000	Whether upper-end resets are server- or client-side
R5	1,600 questions: 100% recoverability, 0 duplicate options, 100% tampered-token rejection. Presentation defect found in `ChoiceList` and fixed	Per-module samples remain bounded; grading trust was not re-measured after the fix

well above the published mean of 68 [2], with 26 of 33 respondents at or above 68 and all three cycles in the same "Good" adjective band. The result that matters most is Cycle 3, the only cycle run with the target population in the target setting: 79.8 overall, rising to 82.5 among the 10 self-identified first-years. Usability did not degrade when the audience moved from experienced reviewers to novices, which is where a tool of this kind is most likely to lose ground.

Two caveats temper that. Dispersion is wide in Cycles 2 and 3 (SD 17.2 and 13.7, each spanning a low outlier and a perfect 100), and internal consistency is weak: Cronbach's α over the ten items is 0.63 pooled and only 0.16 in Cycle 1, far below the $\alpha \approx 0.9$ typical of large SUS samples. Restricted range at $n = 33$ explains this, but it is a reason to treat the SUS figures as indicative rather than precise. At item level, learnability and desirability improved across the cycles while two reverse-worded items moved the wrong way, "very cumbersome" from 1.30 to 2.08 and "too much inconsistency" from 1.60 to 1.85, consistent with scope growing from a single practice module to eight exercised alongside Challenges, Code Space, and the leaderboard (Supplement S12). Cycle 3's mental-demand rating was nonetheless low, median 3 of 10 with 11 of 13 at 5 or below; the single rating of 9 came from the respondent with the lowest SUS score, 57.5, who also selected four checklist issues.

Cycle 3's ten-item battery attached agreement to individual design claims (Supplement S15, Table S15.1 and Figure S15.1). The two visualisation claims are the strongest results in the whole human evaluation: 13 of 13 agreed that highlighted indices and slices helped them reason about string operations (mean 4.69) and 12 of 13 that the step-by-step visualiser made code behaviour easier to understand (4.46), neither drawing a single disagreement. Misconception feedback followed at 11 of 13 (4.46), and the claim closest to this paper's core concept, distinguishing changing a value from reassigning a variable, at 10 of 13 (4.15); the five core pedagogical items average 4.32. The weakest claim is the adaptive one: only 9 of 13 agreed practice felt relevant to what they were struggling with, three disagreed (3.85), and two respondents independently asked for graded difficulty and fuller per-module coverage. The two isiXhosa items are lowest of the ten (3.31 and 3.46). Asked to name one most-helpful feature, respondents chose the visualiser most often (5 of 13), then practice questions (3), feedback and Challenges (2 each), and Code Space (1).

6.2.2 Applied knowledge and the immutability misconception. The knowledge check is the only instrument that does not rely on self-report, and it was administered in substantially the same form in all three cycles (Table 5b, Figure S12.1). Indexing was correct for 33 of 35 participants and case-sensitive comparison for 32 of 35. Slicing was weaker at 30 of 35, and every one of the five wrong answers was the identical off-by-one, `"ytho"` for `word[1:4]`, which is the inclusive-stop model. Cycle 3's added item on whether `text.upper()` mutates its receiver was correct for 11 of 13.

The immutability item is the substantive finding (Figure 3). Pooled over all three cycles only 24 of 35 (69%) correctly identified that `word[0] = "J"` is rejected. Of the 11 who did not, 10 held a model in which no error occurs at all: 7 asserted the assignment succeeds so that `"Python"` silently becomes `"Jython"`, and 3 that nothing happens and nothing is raised; the last expected an error but named `IndexError`. The silent-mutation model appeared in the same nameable form in all three independent cohorts, 2 of 11, 2 of 11, and 3 of 13, and did not diminish with seniority, since the Cycles 1 and 2 respondents were second-year, third-year, and Honours students with at least a year of programming behind them.

This is the evaluation's strongest link between the artefact and the literature it rests on. The system is designed on the premise that immutability errors are conceptual rather than syntactic and so persist unchallenged until the language finally surprises the learner [11,16,19]; the knowledge check supplies direct, locally gathered evidence for that premise. It also explains the gap between 69% on immutability and 94% on indexing: a learner can index correctly for a year while still believing the characters are writeable, because

nothing in ordinary practice contradicts the belief. Two caveats: it is one item, so it measures the stated model rather than a stable conception, and the cohorts differ in seniority, so the three cycles are not a trend. The data support the existence and persistence of the misconception in this population, not a rate for any wider one.

Self-reported confidence rose in all three cycles on every concept measured, from 3.16 to 4.50 in Cycle 1 ($n = 11$, exact Wilcoxon $p = 0.004$), 3.28 to 4.47 in Cycle 2 ($p = 0.008$), and 3.06 to 4.15 in Cycle 3 ($n = 13$, $p = 0.002$); pooled, 27 of 35 rated themselves higher, 7 unchanged, 1 lower. Cycle 3's largest single-concept gain was tracing code step by step, 2.85 to 4.08 ($p = 0.002$), the concept the tracer exists to support, while immutability produced the largest gains in Cycles 1 and 2 from the lowest starting base of any concept. Because both ratings were given after the session this is a perception measure only, and the complete absence of any decrease in Cycles 1 and 3 (0 of 40 and 0 of 65 concept-level pairs) is treated here as a warning of acquiescence rather than reassurance. Cycle 2, with 6 of 50 pairs decreasing, is the more credible pattern: two of those four participants named as still-unclear the very concept whose rating they lowered (Supplement S16).

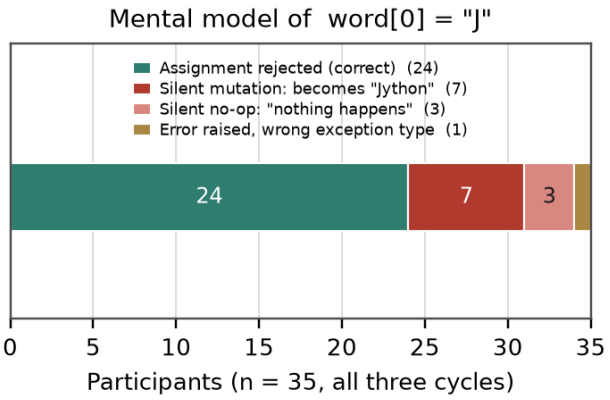


Figure 3: Mental model implied by every participant's answer to the in-place character assignment item, pooled over all three cycles. Ten of the 35 expected no error at all. Per-concept accuracy by cycle is in Table 5b, and the same data plotted per cycle in Supplement S12 (Figure S12.1).

6.2.3 Reported problems, expert review, and localisation. Cycles 1 and 2 collected problems as free text and found the defects; Cycle 3's fixed checklist, built from those same reports, measured how prevalent each remaining problem was with the target population (Table S15.2, Figure S15.2). In Cycle 3, 5 of 13 selected "no major issues". The most prevalent remaining problem is wayfinding, with 5 of 13 unsure what to do next at some point, though one clarified that this was exploration rather than confusion; then session length at 4 of 13, which is partly a study artefact given that the cycle ran directly after a practical. Two findings at 3 of 13 each matter more: participants who did not trust whether their answer had been graded correctly, and participants who found the isiXhosa terminology unclear or inconsistent. Only 1 of 13 reported navigation as confusing and 1 Code Space, both prominent complaints in Cycle 1.

The three peer developers were not uniformly positive, and both their positive and negative findings are specific. All three independently singled out the visualiser, one calling it "very interactive & intuitive to use", and two independently praised the leaderboard. All three also converged on one weakness, the same one the students raised: assessment robustness. One reported answering −1 to an item asking for the index of `"a"` in `"xhosa"` and being marked wrong; the expected answer is 4, so the mark was defensible, but the wording did not signal whether a search-miss convention applied. A second reported that "the marking is not 100% accurate", that some items omitted the expected answer from the options, that negative indexing was not always accounted for, and that some code-writing items displayed the expected output so a learner could satisfy them by printing it verbatim. A third questioned grading path coverage and reported too many similar questions. Error explanations in Challenges were raised once, and a lecturer-facing analytics dashboard was offered as an idea rather than a defect (Supplement S17). The four HCI reviewers converged on the same navigation and feedback-wording issues the student free text raised, while independently praising the visualiser's clarity.

The supervisor's acceptance walkthrough covered 30 checklist items across seven categories. Twenty-nine carried a recorded verdict and all 29 were accepted, one was left blank, and none was marked as requiring a fix before submission. Two were accepted with a corrective comment: Code Space should offer an option to clear the code, and the newline escape must display as a single character. The written assessment recorded that the system meets the proposal's requirements "and has extra functionality". The newline comment bears directly on this contribution, since character-cell rendering comes from the tracer's index resolution (Section 4.2).

On localisation, the single isiXhosa item was answered by 27 of 35 participants, pooled mean 3.59 of 5 with 17 of 27 rating it 4 or 5; the 8 blanks are themselves a signal that not everyone exercised the feature. The free text is sharper than the ratings. One Cycle 1 participant described being able to think in their first language and code directly, without an intermediate translation into English, which is the strongest statement of the feature's value anywhere in the data. Against that, specific defects recur across cycles: phrases identified as isiZulu rather than isiXhosa, boolean literals left in English after switching, and a Cycle 3 respondent having to keep checking the keyword mapping. The 3-of-13 checklist rate did not fall relative to earlier cycles. This is preliminary evidence that composing code in isiXhosa has real value for some learners, and equally direct evidence that keyword-level transpilation does not by itself deliver a coherent bilingual interface.

Supplements S13 and S14 (Tables S13.1 and S14.1) collate the recurring themes across all three cycles and trace each finding to its disposition, stating conservatively which were addressed with evidence, which are partial, and which remain open.

6.3 Discussion

The technical results establish that the backend is reliable enough to carry the intended learning workflow within the scope tested.

Table 5: Human evaluation results. (a) System Usability Scale by cycle; scales with a missing item were excluded, not imputed. (b) Applied knowledge check, participants answering correctly; Cycle 3's marks were independently regraded and agree with the instrument on all 13 respondents. The pooled breakdown of mental models implied by the immutability answers is in Figure S12.1b.

(a) System Usability Scale

Cycle	n	Mean	SD	95% CI	≥ 68
1 (experienced)	10	79.2	6.2	74.8–83.7	9/10
2 (experienced)	10	83.5	17.2	71.2–95.8	7/10
3 (first-year lab)	13	79.8	13.7	71.5–88.1	10/13
Pooled	**33**	**80.8**	**13.0**	**76.2–85.4**	**26/33**
3, first-years only	10	82.5	11.6	74.2–90.8	9/10

(b) Applied knowledge check, correct answers

Cycle	n	Idx	Slice	Immut.	Cmp.
1 (free text)	11	10	9	8	11
2 (free text)	11	11	11	8	11
3 (auto-marked)	13	12	10	8	10
Pooled	**35**	**33**	**30**	**24**	**32**
Pooled %		94	86	69	91

What the human evidence adds beyond confirming that is three things.

The premise is not merely borrowed from the literature. That immutability errors are conceptual rather than syntactic is a claim taken from prior work [11,16,19]; the knowledge check tests it locally, and it holds in three independent cohorts without diminishing with seniority. This matters for the design argument because it justifies the decision that most shaped the backend: resolving character indices inside the tracer, so a learner sees which characters an operation touches rather than only its result, is a direct response to a misconception these participants demonstrably held.

The component the design bet most heavily on is the one four independent instruments endorse. Rating scale, forced choice, confidence gain, and unprompted expert review all converge on the visualiser, and measures with different failure modes agreeing is stronger evidence than any one of them. It is also the payoff for the R2 methodology, since the mechanism producing the endorsed visualisation is the one verified differentially against CPython.

The human and technical evaluations converged on the same weakness from opposite directions. Assessment robustness, not the lessons and not the visualiser, is what Cycle 2's free text, all three peer developers, and 3 of 13 Cycle 3 respondents independently questioned; that convergence prompted the R5 sweep, which located the `ChoiceList` defect. This is the most useful methodological result here, because a formative complaint ("I did not trust the mark") and an automated requirement check identified one defect from opposite ends. It also matters educationally rather than cosmetically: the practice engine converts misconception information into repeated targeted practice, so a defect that makes learners doubt their own marks attacks that mechanism. The matching open item is equally clear from the same data, since adaptive relevance was the weakest claim in the battery and the free text asks for graded difficulty and fuller per-module coverage. R4's saturation is a lesser case of the same pattern: a configuration limit with a demonstrated remedy rather than a flaw in the execution path. The isiXhosa feature remains a secondary enhancement, with the recurring finding that keyword-level transpilation alone does not produce a coherent bilingual interface.

The boundary of all of this bears repeating. The evaluation establishes requirement satisfaction, technical correctness within declared scope, and usability with the target population. It does not establish learning effectiveness: the confidence measure is retrospective and uncontrolled, the knowledge check is one item per concept, and each cycle evaluated a repaired build, so cross-cycle figures track a moving target.

7 Conclusion

This paper presented the backend of String Conundrum, an interactive system for teaching Python strings: sandboxed execution of learner code, an AST-based statement-level tracer that supplies the character indices the visualisations consume, procedurally generated adaptive practice across eight modules, authentication and persistent progress, challenge and leaderboard data, and optional English–isiXhosa keyword and runtime-error support.

All five measurable requirements were satisfied within their declared scope: R1 contained every case in its threat set, R2 matched observed CPython behaviour across the supported AST subset, R3 preserved both text and executable behaviour, R4 met the two-second target at classroom-scale concurrency with saturation beyond that traced to worker count rather than the execution path, and R5 achieved fully recoverable answers with complete tampered-token rejection while also exposing and fixing a grading-presentation defect. FR-6 was confirmed against the production database.

The principal contribution is the demonstration that string-specific pedagogical data can be produced as a verified by-product of execution tracing rather than reconstructed in the interface, and that one shared practice core can serve eight concept areas while remaining independently testable. The methodological contribution is the convergence result: a formative usability complaint and an automated requirement experiment independently located the same defect, which is stronger evidence about the artefact than either alone.

The evidence establishes requirement satisfaction, technical correctness within the scope tested, and usability with the intended first-year population. It does not establish improved learning outcomes; that requires the controlled pre-/post-test study that remains outstanding. The most important future work is therefore that study, followed by the two open items the evaluation itself identifies: surfacing graded difficulty so adaptive practice is perceived as relevant, and closing the gap between keyword-level transpilation and a coherent bilingual interface.

References

[1] Amjad Altadmri and Neil C. C. Brown. 2015. 37 Million Compilations: Investigating Novice Programming Mistakes in Large-Scale Student Data. In *Proceedings of the 46th ACM Technical Symposium on Computer Science Education (SIGCSE)*. ACM, New York, NY, USA, 522–527. https://doi.org/10.1145/2676723.2677258.

[2] Aaron Bangor, Philip T. Kortum, and James T. Miller. 2008. An Empirical Evaluation of the System Usability Scale. *International Journal of Human–Computer Interaction* 24, 6 (2008), 574–594. https://doi.org/10.1080/10447310802205776.

[3] John Brooke. 1996. SUS: A Quick and Dirty Usability Scale. In *Usability Evaluation in Industry*, Patrick W. Jordan, Bruce Thomas, Bernard A. Weerdmeester, and Ian L. McClelland (Eds.). Taylor & Francis, London, 189–194. https://doi.org/10.1201/9781498710411.

[4] Albert T. Corbett and John R. Anderson. 1994. Knowledge Tracing: Modelling the Acquisition of Procedural Knowledge. *User Modeling and User-Adapted Interaction* 4, 4 (1994), 253–278. https://doi.org/10.1007/BF01099821.

[5] Philip J. Guo. 2013. Online Python Tutor: Embeddable Web-Based Program Visualization for CS Education. In *Proceedings of the 44th ACM Technical Symposium on Computer Science Education (SIGCSE '13)*. ACM, New York, NY, USA, 579–584. https://doi.org/10.1145/2445196.2445368.

[6] Sandra G. Hart and Lowell E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. In *Human Mental Workload*, Peter A. Hancock and Najmedin Meshkati (Eds.). North-Holland, Amsterdam, 139–183. https://doi.org/10.1016/S0166-4115(08)62386-9.

[7] Felienne Hermans. 2020. Hedy: A Gradual Language for Programming Education. In *Proceedings of the 2020 ACM Conference on International Computing Education Research (ICER '20)*. ACM, New York, NY, USA, 259–270. https://doi.org/10.1145/3372782.3406262.

[8] Michael C. Hughes, Matthew C. Jadud, and Ma. Mercedes T. Rodrigo. 2010. String Formatting Considered Harmful for Novice Programmers. *Computer Science Education* 20, 3 (2010), 201–228. https://doi.org/10.1080/08993408.2010.507335.

[9] Cruz Izu and Amali Weerasinghe. 2020. Assessing CS1 Design Skills with a String Manipulation Task. In *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE)*. ACM, New York, NY, USA, 432–438. https://doi.org/10.1145/3341525.3387382.

[10] Johan Jeuring, Alex Gerdes, and Bastiaan Heeren. 2012. Ask-Elle: A Haskell Tutor – Demonstration. In *21st Century Learning for 21st Century Skills: Proceedings of the 7th European Conference on Technology Enhanced Learning (EC-TEL 2012)*, Lecture Notes in Computer Science, Vol. 7563. Springer, Berlin, Heidelberg, 453–458. https://doi.org/10.1007/978-3-642-33263-0_42.

[11] Kuang-Chen Lu and Shriram Krishnamurthi. 2024. Identifying and Correcting Programming Language Behavior Misconceptions. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1, Article 106 (2024). https://doi.org/10.1145/3649823.

[12] Bradley N. Miller and David L. Ranum. 2012. Beyond PDF and ePub: Toward an Interactive Textbook. In *Proceedings of the 17th ACM Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE '12)*. ACM, New York, NY, USA, 150–155. https://doi.org/10.1145/2325296.2325335.

[13] Iain Milne and Glenn Rowe. 2002. Difficulties in Learning and Teaching Programming: Views of Students and Tutors. *Education and Information Technologies* 7, 1 (2002), 55–66. https://doi.org/10.1023/A:1015362608943.

[14] Andrés Moreno, Niko Myller, Erkki Sutinen, and Mordechai Ben-Ari. 2004. Visualizing Programs with Jeliot 3. In *Proceedings of the Working Conference on Advanced Visual Interfaces (AVI '04)*. ACM, New York, NY, USA, 373–376. https://doi.org/10.1145/989863.989928.

[15] Roy D. Pea. 1986. Language-Independent Conceptual Bugs in Novice Programming. *Journal of Educational Computing Research* 2, 1 (1986), 25–36. https://doi.org/10.2190/689T-1R2A-X4W4-29J2.

[16] Yizhou Qian and James Lehman. 2017. Students' Misconceptions and Other Difficulties in Introductory Programming: A Literature Review. *ACM Transactions on Computing Education* 18, 1, Article 1 (2017). https://doi.org/10.1145/3077618.

[17] Anthony Robins, Janet Rountree, and Nathan Rountree. 2003. Learning and Teaching Programming: A Review and Discussion. *Computer Science Education* 13, 2 (2003), 137–172. https://doi.org/10.1076/csed.13.2.137.14200.

[18] Jeff Sauro and James R. Lewis. 2016. *Quantifying the User Experience: Practical Statistics for User Research* (2nd ed.). Morgan Kaufmann, Cambridge, MA. https://www.oreilly.com/library/view/quantifying-the-user/9780128025482/.

[19] Juha Sorva. 2012. *Visual Program Simulation in Introductory Programming Education*. Ph.D. Dissertation. Aalto University, Finland. https://urn.fi/URN:ISBN:978-952-60-4626-6.

[20] Juha Sorva, Ville Karavirta, and Lauri Malmi. 2013. A Review of Generic Program Visualization Systems for Introductory Programming Education. *ACM Transactions on Computing Education* 13, 4 (2013), 1–64. https://doi.org/10.1145/2490822.

[21] Kurt VanLehn. 2011. The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems. *Educational Psychologist* 46, 4 (2011), 197–221. https://doi.org/10.1080/00461520.2011.611369.

[22] Frank Wilcoxon. 1945. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1, 6 (1945), 80–83. https://doi.org/10.2307/3001968.

AI Usage Statement

I used two generative-AI tools, ChatGPT and GitHub Copilot, for code-related assistance and for reviewing and improving my own writing. GitHub Copilot was used primarily as an in-editor coding assistant during backend development, including drafting boilerplate, suggesting test cases, and helping debug the sandbox, tracer, and practice-engine modules, while ChatGPT was used more for prose: restructuring paragraphs, tightening argument flow, and checking that terminology was used consistently across the paper. Neither tool was used to generate the experimental results, the human evaluation data, or the analysis reported in this paper; all figures, statistics, and coded responses come from the author's own scripts and the raw data accompanying the submission. I critically reviewed every AI-assisted suggestion before accepting it, independently verified the claims, citations, and numerical results included in this paper against their original sources, and take full responsibility for the final submission, including any remaining errors. No other generative-AI tools were used.

Supplementary Information

S1 Index and How to Read This Supplement

The material below supports and documents the main paper but is not required to understand the contribution, the design, the methodology, the principal results, the limitations, or the conclusions.

It is organised into five parts, and within each part the sections appear in the order the main paper first cites them, so this supplement can be read either front-to-back or by following a citation from the paper. Parts A and B expand the design and the implementation contracts; Part C holds the technical evaluation data, ordered by requirement; Part D holds the human evaluation data; and Part E collects the full-size diagrams, placed last only because each occupies a whole page. Table S1.1 is the index: every entry names the section of the main paper it supports and what it contains.

Table and figure numbers restart within each supplement section, so Table S4.2 is the second table of Supplement S4.

Table S1.1: Index to the Supplementary Information. "Cited from" is the section of the main paper that first refers to the material.

	Supplement	Cited	Contents
Part A	*Background and Design*		
S2	Comparison with related systems	§2	Feature matrix over seven comparable systems
S3	Functional and non-functional requirements	§3	Expanded FR-1–FR-6 and NFR-1–NFR-6
Part B	*Implementation Contracts*		
S4	Tracer AST support contract	§4.2	Accepted statement and expression forms; what is out of scope
S5	English–isiXhosa keyword mapping	§4.3	Representative subset of the shared keyword table
S6	Practice-core structure	§4.4	Core/module boundary and per-layer responsibility
S7	API endpoints	§4.5	Every endpoint owned by this contribution, with auth status
Part C	*Technical Evaluation Data*		
S8	Automated test breakdown	§6	All 274 tests by file
S9	Transpiler fidelity (R3)	§6	Textual and behavioural round-trip results, $n = 18$
S10	Performance measurements (R4)	§6	Nine concurrency levels, plus the replicated worker-count experiment
S11	R5 module-level detail	§5	Per-module sweep over 1,600 generated questions
Part D	*Human Evaluation Data*		
S12	Instruments, scoring and reliability	§5.2	SUS scoring and α, per-item means, coding rubric, coded answers
S13	Theme breakdown	§6.2	Recurring themes across all three cycles and the expert reviews
S14	Formative findings and disposition	§6.2	Every finding traced to addressed, partial, or open
S15	Cycle 3 detail	§6.2	Design-claim battery, issue checklist, isiXhosa item
S16	Confidence detail	§6.2	Concept-level pre–post statistics for all three cycles
S17	Expert peer review and acceptance	§6.2	Cross-team review record and the supervisor checklist
Part E	*Full-Size System Diagrams*		
S18	System diagrams	§4	Tracer, sandbox threat model, schema, and request-lifecycle diagrams

Part A Background and Design

Supports Sections 2 and 3: the feature-by-feature comparison behind the research gap, and the full requirement statements.

S2 Comparison with Related Systems

Table S2.1 expands the synthesis of Section 2 into a feature-by-feature comparison. The gap it documents is the combination the main paper argues for: none of the related systems pairs string-specific step-by-step visualisation with misconception-targeted adaptive practice.

S3 Functional and Non-Functional Requirements

Table S3.1 expands the functional requirements introduced in Section 3. Table S3.2 expands the non-functional requirements introduced in the same section.

Table S2.1: Feature comparison between String Conundrum and related programming-education systems and approaches.

System / Approach	Program execution	Step-by-step visualisation	String-specific teaching	Adaptive / targeted practice	Misconception-aware feedback	Localisation / multilingual support	Main limitation relative to String Conundrum
Online Python Tutor [5]	✓	✓	×	×	×	Limited	Excellent general Python execution visualisation, but not organised around string concepts, adaptive practice, or string-specific misconceptions.
Jeliot 3 [14]	✓	✓	×	×	×	×	Designed to visualise program data and control flow for novices, but is a general program-visualisation system rather than a string-focused tutoring environment.
SMoL Tutor [11]	Limited / tutor-controlled	✓	×	✓	✓	×	Strong misconception diagnosis and correction, but focuses on general programming-language semantics and assumes prior programming knowledge rather than teaching Python strings specifically.
Hedy [7]	✓	Limited	×	Structured progression	Error-oriented	✓	Reduces syntax difficulty through a gradual teaching language and supports localisation, but does not focus on detailed Python string semantics or execution-state visualisation.
Runestone / ActiveCode [12]	✓	Some interactive components	×	Some exercise support	General feedback	Limited	Integrates executable code, textbooks, assignments and grading, but is a broad interactive learning platform rather than a targeted system for string misconceptions.
Ask-Elle [10]	✓ / solution checking	×	×	✓	✓	×	Provides automated feedback and hints during Haskell programming exercises, but focuses on functional programming and strategy-based solution development rather than Python strings or execution visualisation.
Generic Intelligent Tutoring Systems [21]	Varies	Varies	×	✓	✓	Varies	Strong adaptive-learning foundations, but require a domain model specific to the concepts being taught; they do not directly provide a Python-string curriculum or execution tracer.
String Conundrum	✓	✓**, string-focused**	✓	✓	✓	✓ **English–isiXhosa**	Currently validated primarily for technical correctness and formative usability; learning effectiveness has not yet been established through a controlled pre-/post-test study.

Table S3.1: Functional requirements (expanded).

Label	Requirement	Description
FR-1	Authentication	Verify a learner's Firebase ID token on every protected route, upsert a corresponding row into the `users` table on first contact, and support two backend-issued token paths (guest and display-name-only) that remove sign-up friction for usability testing.
FR-2	Sandboxed code execution	Accept submitted Python, or, after client-side transpilation, isiXhosa-keyword Python, run it in an isolated subprocess with a fixed timeout, and capture stdout and stderr for both free exploration and challenge grading.
FR-3	Execution tracing	Reconstruct a step-by-step, statement-level trace of a supported program from its AST, including resolved string indices and slices, loop-iteration context, and boolean-condition substitution.
FR-4	Bilingual transpilation	Provide a deterministic, bidirectional keyword-substitution mapping between English and isiXhosa Python source, applied client-side before any code reaches the backend.
FR-5	Adaptive practice generation	Generate a procedurally-composed practice question (template, string, kind, and format) per module on request, grade it via a signed, stateless token, and bias template selection toward a learner's recently-missed concepts when supplied.
FR-6	Data persistence	Store challenges, test cases, per-module and per-section progress, challenge and exercise completions, and leaderboard standing in PostgreSQL, with a local JSON-file fallback for testing and offline development.

Table S3.2: Non-functional requirements (expanded).

Label	Requirement	Description
NFR-1	Performance	Keep `POST /api/execute`'s response time under the project's two-second target through at least the concurrency levels ordinary classroom usage would produce.
NFR-2	Security	Execute untrusted code with process isolation, a wall-clock timeout, a restricted-builtins dictionary blocking imports and filesystem access, and a POSIX memory cap where the platform supports it.
NFR-3	Correctness	Every trace, transpiled program, and generated practice question must be verifiable against an independent source of ground truth, which is real Python execution, a graded test case, or a recomputed answer key, rather than merely internally consistent.
NFR-4	Reliability	Degrade predictably. An unrecognised runtime error still returns a usable response, falling back to raw, transpiler-substituted English rather than failing outright, and a tampered or expired practice-engine token is rejected with a clear error rather than mis-graded.
NFR-5	Extensibility	Let a new practice module reuse the shared token-signing, tier, and adaptive-selection machinery (`practice_engine_core.py`) by implementing only its own templates and question kinds, and let a new isiXhosa keyword or error template be added to one shared table without touching the sandbox, tracer, or route layer.
NFR-6	Maintainability	Keep each concern, which is auth, sandbox, tracer, transpiler support, error translation, each practice engine, and each data model, in its own module behind a single database connection layer, so components can be modified and tested independently.

Part B Implementation Contracts and Interfaces

Supports Section 4: the exact boundaries of what each backend component supports, in the order the implementation section introduces them.

S4 Tracer AST Support Contract

Table S4.1 states the supported subset referred to in Section 4.2. It exists so that the R2 correctness claim has a defined scope: the final row names what the tracer deliberately does not model, and no claim in this paper extends past it.

Table S4.1: Tracer AST support contract used for correctness claims.

Category	Accepted forms
Top-level statements	`Assign`, `AugAssign`, `Expr` (including `print(...)` and string-method calls), `For`, `While`, `If`.
Expression forms used in tracing	`Name`, `Constant`, `BinOp`, `BoolOp`, `UnaryOp`, `Compare`, `Call`, `Attribute`, `Subscript` (index and slice).
String-specific handling	Character-index highlighting for index/slice access, negative-index normalisation, bounded slice highlight ranges, and condition substitution for loop/branch visualisation.
Explicitly out of scope for correctness claims	Imports, function/class definitions, exception-handling blocks, context managers, comprehensions, generators, and other full-language constructs not modelled as traced execution events.

S5 English–isiXhosa Keyword Mapping

Table S5.1 shows a representative subset of the keyword table described in Section 4.3. The full table covers roughly sixty Python keywords and builtins. It is generated from the same `KEYWORD_PAIRS` array that drives both the transpiler and the student-facing reference page, so the two cannot drift out of sync.

Table S5.1: A representative subset of the English–isiXhosa keyword map.

Category	Python	isiXhosa
Control flow	if / elif / else	ukuba / okanye ukuba / ngaphandle
Control flow	for / while	kwi / ngelixa
Functions	def / return	chaza / buyisa
Exceptions	try / except	zama / bamba
Boolean ops	not / and / or	hayi / kwaye / okanye
Literals	True / False / None	Ewe / Hayi / Lutho
Builtins	print / len / range	bhala / ubude / ithuba
Builtins	str / int / list	isigama / inombolo / uluhlu

S6 Practice-Core Structure

Table S6.1 shows where the boundary between the shared core and a module engine falls. This is the concrete form of the NFR-5 extensibility claim in Section 4.4: everything in the first row is written once, and a new module supplies only the two rows below it.

Table S6.1: Shared practice-core structure and module boundary.

Layer / module	Responsibility
`practice_engine_core.py`	Token signing and verification, tier handling, weak-concept weighting helpers, shared answer-normalisation and validation utilities.
`*_engine.py` per module	Public `generate()` and `grade()` interface for one concept area; delegates common mechanics to the core and defines module-specific question kinds and grading logic.
`*_templates.py` per module	Concept-specific template catalogue, tier tagging, and scenario builders used by each module engine.
Route layer (`routes.py`)	Uniform API surface for all modules; dispatches to engine `generate()` / `grade()` without duplicating concept logic.

S7 API Endpoints Owned by This Contribution

Table S7.1 lists every endpoint owned by this contribution, with its authentication requirement, expanding the API boundary described in Section 4.5. One endpoint is deliberately absent: `/api/exercises/{module}`, the static bank, routes into the project partner's misconception-feedback system and is therefore out of scope here, except where it calls back into the author's own progress-recording functions on a correct answer.

Table S7.1: Backend API endpoints owned by the author's contribution.

Method & path	Auth	Purpose
POST /api/execute	No	Run submitted code in the sandbox, and return stdout, stderr, and a full trace.
POST /api/debug	No	Return the tracer's step list for code without also running it.
POST /api/visualize	No	Render a single string expression into indexed, highlighted character cells.
GET /api/challenges	No	List all coding challenges, with solutions and tests stripped.
GET /api/challenges/{id}/solution	No	Return one challenge's reference solution on demand.
POST /api/challenges/run	Yes	Grade a submission against a challenge's stored tests, and award stars on a full pass.
GET /api/progress/{uid}	Yes	Return a learner's section progress, unlock state, and leaderboard preferences, for the own account only.
POST /api/modules/{id}/sections/{id}/complete	Yes	Mark a module section as reached.
PATCH /api/settings	Yes	Update unlock-bypass and leaderboard opt-in and anonymity preferences.
GET /api/leaderboard	No	Return the top-scoring opted-in learners.
POST /api/auth/guest	No	Issue a Firebase custom token for a fresh anonymous session.
POST /api/auth/username	No	Issue a Firebase custom token for a stable, display-name-only account.
GET /api/exercises/{module}/engine/next	No	Generate one procedural practice question for the given module and tier, biased toward recent misconceptions. It is registered identically for all eight modules: intro, indexing, slicing, looping, comparison, methods, immutability, and formatting.
POST /api/exercises/{module}/engine/answer	Yes	Verify a question's signed token and grade the submission, and award stars on a correct answer. It uses the same eight-module registration as above.

Part C Technical Evaluation Data

Supports Sections 5 and 6, ordered by requirement: the automated suite, then R3, R4 and R5. Each table is the full measurement behind one row of Table 4.

S8 Automated Test Breakdown by File

Table S8.1 decomposes the 274 passing tests reported in Section 6. The split matters for interpretation: 254 of the 274 exercise the eight practice engines, and only 20 cover the API and error translation, so the suite is strong evidence about generation and grading and thinner evidence about the route layer.

Table S8.1: Automated backend test results, by file, re-collected directly from the current codebase.

Test file	Tests	Passed
test_api.py (auth, execution, progress endpoints)	12	12
test_error_translation.py (isiXhosa error templates)	8	8
test_intro_engine.py	31	31
test_indexing_engine.py	31	31
test_slicing_engine.py	58	58
test_slicing_engine_api.py	9	9
test_looping_engine.py	24	24
test_comparison_engine.py	25	25
test_methods_engine.py	26	26
test_immutability_engine.py	21	21
test_formatting_engine.py	29	29
Total	**274**	**274**

S9 Transpiler Fidelity Detail (R3)

Table S9.1 separates the two equivalences R3 checks. Textual round-trip fidelity does not imply behavioural equivalence, so both were measured: the third and fourth rows re-execute each round-tripped solution against the challenge's own stored tests rather than comparing source text.

Table S9.1: Transpiler fidelity over the challenge-solution corpus (n = 18). Textual round-trip fidelity and preservation of executable behaviour were checked separately, because the first does not imply the second.

Metric	Result
Unit tests passed	10 / 10
Exact-text round trip (En, Xh, En)	18 / 18 (100%)
Round-tripped solution passes its own tests	18 / 18 (100%)
Identical pass/fail behaviour to the original	18 / 18 (100%)

S10 Performance Measurements (R4)

Two experiments are reported. Table S10.1 is the sweep across nine concurrency levels plotted in Figure 2, and Table S10.2 is the replicated follow-up that isolates worker count. The second is what supports the interpretation in Section 6 that saturation is a configuration limit rather than a property of the execution path.

Table S10.1: `POST /api/execute` against the live deployment at increasing concurrency.

Conc.	Req.	Fail %	p50 (ms)	p95 (ms)	Thr. (req/s)
5	100	0.00	54.5	75.3	85.28
10	100	0.00	113.4	148.0	86.39
20	100	0.00	220.8	242.6	86.58
50	100	0.00	445.0	557.8	88.15
100	200	0.00	1,040.8	1,119.5	90.27
300	600	0.00	3,265.8	3,649.7	85.26
500	1,000	0.00	4,859.1	5,228.9	96.76
800	1,600	48.12	4,086.4	7,553.3	96.75
1,000	2,000	57.90	4,635.1	8,267.2	89.27

A compact replicated benchmark was then run to estimate variability and to isolate the effect of worker count, since a single run at each level cannot distinguish a capacity limit from an artefact. Table S10.2 reports it. At four workers, three repeated runs at 5 concurrent requests gave a mean p95 of 344.9 ms with 28.14 req/s and zero failures; at 100 concurrent requests mean p95 rose to 3.44 s with 29.80 req/s; and at 300 it rose to 9.68 s with 31.60 req/s, so throughput stayed almost flat while latency grew roughly in proportion to load. The worker comparison at fixed concurrency 100 is the decisive part: two workers gave 30.28 req/s at 3.47 s p95 and four gave 29.63 req/s at 3.95 s, while eight gave 57.79 req/s at 1.76 s. Throughput therefore does not improve between two and four workers but nearly doubles at eight, which is the signature of a worker-limited deployment rather than of an execution path that scales poorly. The absolute latencies in this replicated run are

higher than those in Table S10.1 because the two experiments used different client-side request pacing; only the within-experiment comparisons should be read across rows.

Table S10.2: Replicated benchmark of `POST /api/execute`: variability at fixed worker count, and the effect of worker count at fixed concurrency.

Workers	Conc.	Mean p95	Thr. (req/s)	Fail %
Variability: three repeated runs per level				
4	5	344.9 ms	28.14	0.00
4	100	3.44 s	29.80	N/A
4	300	9.68 s	31.60	N/A
Worker count at fixed concurrency 100				
2	100	3.47 s	30.28	N/A
4	100	3.95 s	29.63	N/A
8	100	**1.76 s**	**57.79**	N/A

S11 R5 Module-Level Details

Table S11.1 gives the per-module result of the 1,600-question sweep described in Section 5, 200 questions per engine. Reporting it per module rather than pooled is deliberate: a pooled figure would hide a single weak engine, and the per-module view is what shows the shared core behaving uniformly across all eight concept areas.

Table S11.1: Cross-module R5 sweep summary (corrected run).

Module	Questions	Recoverability	Duplicate options
Intro	200	100%	0
Indexing	200	100%	0
Slicing	200	100%	0
Looping	200	100%	0
Comparison	200	100%	0
Methods	200	100%	0
Immutability	200	100%	0
Formatting	200	100%	0
Total	**1,600**	**100%**	**0**

The R5 negative-path checks also included tampered-token submissions and malformed payload handling. In the corrected run, tampered-token rejection was 100% (1,600/1,600). The earlier Immutability recoverability anomaly was traced to an evaluator-side delimiter bug and was resolved in the rerun.

Part D Human Evaluation Data

Supports Sections 5.2 and 6.2: instruments and reliability first, then the thematic and corrective record, then the per-cycle quantitative detail, then the expert reviews.

S12 Human Evaluation: Instruments, Scoring, and Reliability

All three cycles administered the ten-item System Usability Scale, a retrospective paired five-point confidence measure, an objectively markable applied knowledge check, a single five-point isiXhosa-helpfulness item, and free-text prompts. Cycle 3 additionally administered a ten-item design-claim agreement battery, a one-to-ten mental-demand rating, a fixed issue checklist, and a single-choice most-helpful-feature question.

SUS was scored with the standard formula: for the five positively worded items the contribution is the response minus one, for the five reverse-worded items it is five minus the response, and the sum is multiplied by 2.5. Any scale with a missing item was excluded rather than imputed, which removes one respondent in Cycle 1 and one in Cycle 2. Table S12.1 lists every retained score and Table S12.2 the per-item means.

Internal consistency of the ten SUS items, computed as Cronbach's α on the reverse-scored responses, is 0.16 for Cycle 1, 0.58 for Cycle 2, 0.69 for Cycle 3, and 0.63 pooled over the 33 complete scales. These are below the 0.7 convention and well below the values typically reported for SUS in large samples. The most likely cause is the combination of small n with a restricted score range, which is severe in Cycle 1 where all ten scores fall between 67.5 and 85.0. The SUS results in Section 6.2 should be read as indicative rather than as precise interval estimates.

Figure S12.2 shows the distribution of every individual score and the underlying item pattern that the main paper reports in summary, and Figure S12.1 plots the applied knowledge check per cycle alongside the pooled mental-model breakdown that Figure 3 gives in the main paper.

Confidence pairs were tested with the two-sided Wilcoxon signed-rank test [22]. Because $n \leq 13$ per cycle, the null distribution was obtained exactly by enumerating all 2^n sign assignments of the non-zero difference ranks rather than by normal approximation; zero differences were dropped. The pooled participant-level test, at $n = 35$, used a 400,000-resample sign-flip randomisation with a fixed seed. Effect sizes are the matched-pairs rank-biserial correlation and d_z. No correction for multiple comparisons was applied, because the concept-level tests are not independent and are reported as descriptive support for direction of change rather than as confirmatory tests.

Coding rubric for the immutability item

Cycles 1 and 2 collected free-text answers to "what happens when `word[0] = "J"` is executed, and why?" for `word = "Python"`. Answers were coded on three levels, because the informative distinction is which mental model an answer implies rather than simply whether it is right:

Table S12.1: Every retained SUS score, by cycle, with the median, adjective anchor [2] and curved grade [18] for each. All three cycles fall in the same adjective band.

Cycle	Individual SUS scores	Med.	Adj.	Grade
Cycle 1	82.5, 67.5, 85.0, 82.5, 75.0, 82.5, 85.0, 70.0, 80.0, 82.5	82.5	Good	B+
Cycle 2	85.0, 67.5, 100.0, 97.5, 67.5, 47.5, 87.5, 97.5, 95.0, 90.0	88.8	Good	A
Cycle 3	57.5, 80.0, 90.0, 92.5, 62.5, 87.5, 92.5, 100.0, 70.0, 72.5, 65.0, 92.5, 75.0	80.0	Good	B+
Pooled	all 33 complete scales	82.5	Good	B+

Table S12.2: Mean raw SUS item responses (1–5). Items marked (R) are reverse-worded, so lower is better.

Item	C1	C2	C3
1. Would use frequently	3.80	4.50	4.46
2. Unnecessarily complex (R)	1.50	1.90	1.77
3. Easy to use	4.10	4.60	4.08
4. Needs technical support (R)	1.60	1.80	1.69
5. Functions well integrated	4.00	4.10	4.15
6. Too much inconsistency (R)	1.60	1.60	1.85
7. Most would learn quickly	3.70	4.80	4.38
8. Very cumbersome (R)	1.30	1.90	2.08
9. Felt confident	4.10	4.30	4.38
10. Much to learn first (R)	2.00	1.70	2.15

- **Correct**: states that the assignment fails, raises an error, or that a string's characters cannot be assigned to. Naming the exception was not required.
- **Silent no-op**: recognises that the string is not modified but asserts that no error is raised.
- **Silent mutation**: asserts that the assignment succeeds and `"Python"` becomes `"Jython"`.

Cycle 3 replaced the free-text item with fixed options whose distractors encode these same models directly, which is why the Cycle 3 counts are verifiable without coder judgement. Table S12.3 lists every answer that was not coded correct, so the classification can be audited in full.

S13 Human Evaluation Theme Breakdown

Table S13.1 pools the free text from all three cycles with the expert reviews and reports each theme once, with the evidence for it drawn from wherever it appeared. The first two rows are the two strongest findings in the whole human evaluation, one positive and one negative; the rest are ordered by how often they recurred.

S14 Formative Findings and Their Disposition

Tables S14.1 and S14.2 trace each recurring finding to what was done about it and to the evidence for that claim. The record is split by disposition so that neither half hides the other: Table S14.1 holds the findings that were addressed or partially addressed, and

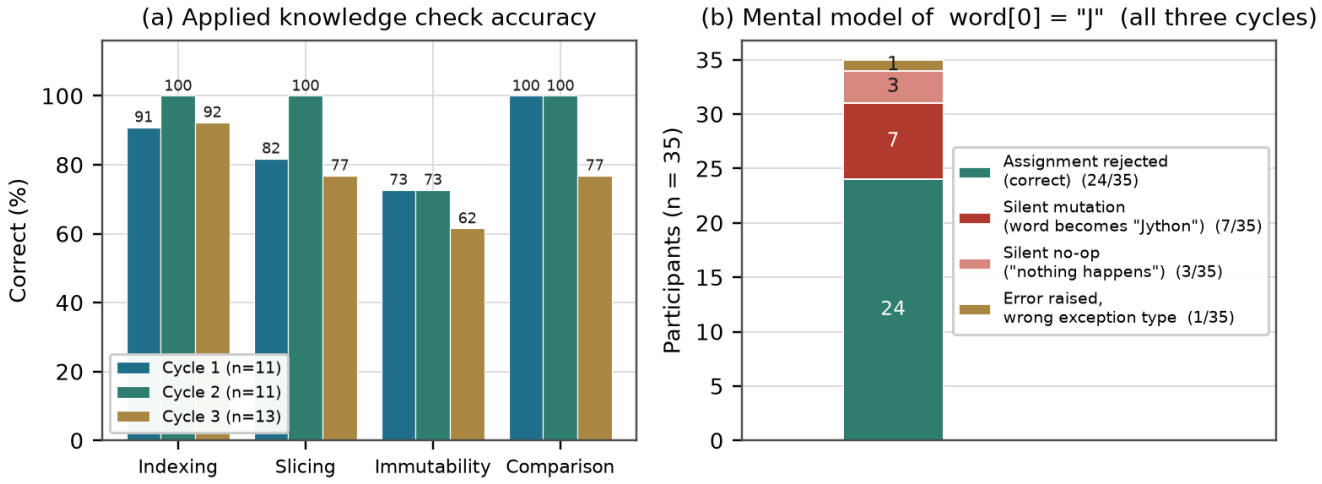


Figure S12.1: Applied knowledge check. Part (a) gives per-concept accuracy in each cycle; the four concepts were assessed in substantially the same form throughout, as free text in Cycles 1 and 2 and as auto-marked items with misconception distractors in Cycle 3. Part (b) pools all 35 participants' answers to the in-place character assignment item and classifies the mental model each answer implies.

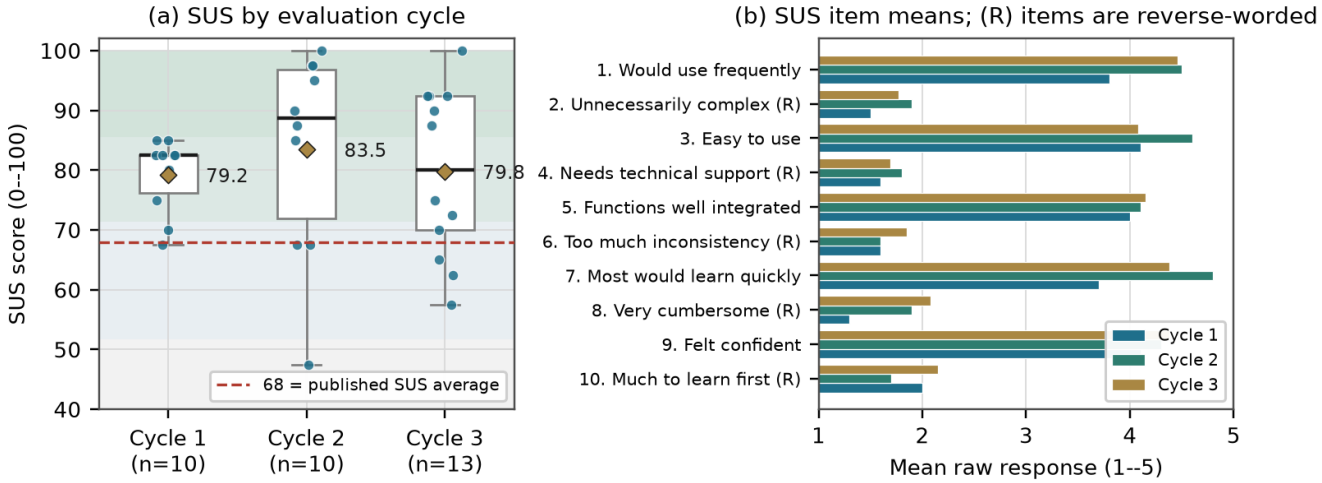


Figure S12.2: System Usability Scale results across the three evaluation cycles. Part (a) shows every individual score, with the box summarising each cycle, the diamond marking the mean, the dashed line at the published SUS mean of 68, and the shaded bands giving the adjective anchors of Bangor et al. [2]. Part (b) shows the mean raw response to each of the ten items; the six positively worded items are better when higher and the four reverse-worded items, marked (R), are better when lower. The two items that improved most across the cycles are "most people would learn to use this tool very quickly" (3.70 to 4.80 and 4.38) and "I would like to use this tool frequently" (3.80 to 4.50 and 4.46).

Table S14.2 holds those that remain open, are the project partner's to fix, or fall outside the agreed scope.

Disposition is stated conservatively. *Addressed* is used only where this paper reports independent evidence, *partial* where a mechanism exists but the underlying request is not fully met, and *open* where the finding stands. Because each cycle evaluated a repaired build, the later cycles function as a partial re-test of the earlier findings, and this is noted where it applies.

S15 Human Evaluation: Cycle 3 Detail

Cycle 3 ran on 14–15 September 2026 in a University of Cape Town Computer Science laboratory venue directly after a first-year practical session. Thirteen students responded; 10 self-identified as

Table S12.3: All non-correct answers to the in-place character assignment item, across the three cycles ($n = 11$ of 35). Cycles 1 and 2 are verbatim free text; Cycle 3 entries are the fixed option selected.

Cyc.	Answer given	Coded model
1	"The first letter of the word variable changes from P to J so now the word variable contains *Jython*"	Silent mutation
1	"The first letter P in Python will be changed to J, the word will now be Jython"	Silent mutation
1	"Nothing happens since the second line can't actually change the characters in the string held in the word variable."	Silent no-op
2	"Jython. Index 0 *P* is changed to *J*"	Silent mutation
2	"It replaces the first element *P* with *J*"	Silent mutation
2	"Nothing happens because a string is immutable"	Silent no-op
3	*word* becomes `"Jython"`.	Silent mutation
3	*word* becomes `"Jython"`.	Silent mutation
3	*word* becomes `"Jython"`.	Silent mutation
3	Nothing changes and no error is raised.	Silent no-op
3	It raises an `IndexError`.	Error, wrong type

current UCT first-years and all 13 reported prior exposure to introductory Python. Modules completed in the session were String Slicing (10 of 13), Introduction to Strings (7), String Looping (6), and other or additional modules (3). Two participant codes collided, so it is possible that two responses came from one person; their confidence and knowledge responses differ substantially, and both are retained.

Table S15.3 reports the single isiXhosa-helpfulness item across all three cycles rather than Cycle 3 alone, since it is the one localisation measure the cycles have in common. The blank counts are retained in the table because the number of participants who did not answer is itself informative.

S16 Human Evaluation: Confidence Detail

Table S16.1 gives the concept-level confidence results for all three cycles and Figure S16.1 plots them. Both ratings in each pair were given after the session on the same questionnaire, so this is a retrospective pre–post measure without a control condition; it is reported as a perception measure and no learning claim is made from it. The *n* column varies within a cycle because some participants left individual concept ratings blank and no imputation was applied. The looping concept in Cycle 2 was added mid-cycle, so only 6 of 11 respondents saw it.

Individual ratings decreased in 0 of 40 concept-level pairs in Cycle 1 and 0 of 65 in Cycle 3. Cycle 2 is the only cycle with decreases, at 6 of 50 pairs, from four participants. The complete absence of decreases in the other two cycles is better read as evidence of a ceiling or acquiescence effect than as evidence of a uniformly positive experience.

S17 Expert Peer Review and Supervisor Acceptance

Cross-team peer review

Three developers of PyTutor, a Python-learning tool built by a different team for a comparable problem, reviewed String Conundrum between 13 and 18 August 2026. The review was deliberately unscored and had no rubric: the reviewers were asked what they tried first, what they thought was a good design decision, what they would have built differently, anything confusing or broken, anything their own system does differently that is worth knowing, and an overall impression. Table S17.1 summarises what each reviewer reported.

Supervisor acceptance walkthrough

The project supervisor conducted a single final walkthrough of the deployed production build at `string-1.cs.uct.ac.za` on 10 September 2026, following the regular weekly reviews held throughout the project. The purpose was to record formal acceptance of the finished system feature by feature in one sitting. Verdicts were recorded as accepted, accepted with comment, requires a fix before submission, or not applicable. Table S17.2 gives the outcome by category.

Twenty-nine of the 30 checklist items carried a recorded verdict and all 29 were accepted. One item, sign-up via email link, was left blank. No item was marked as requiring a fix before submission. Two items were accepted with a corrective comment, recorded in Table S14.1: Code Space should offer an option to clear the code, and the newline escape must be displayed as a single character. In answer to whether the system meets the requirements set out in the project proposal, the supervisor recorded "Yes, and has extra functionality. It is nice & easy to use and very helpful. Good use of colour to highlight different elements." The general comment was "Excellent work." The acceptance statement was signed and dated 10 September 2026.

Table S13.1: Recurring themes across the free-text responses and expert reviews from all three cycles. The first two rows are the strongest recurring positive and negative findings respectively.

Theme	Representative evidence
Visualisation	The single most consistent positive finding. Named unprompted in 7 of 11 Cycle 1 free-text responses, credited with reducing the mental load of tracking character positions; endorsed by 13 of 13 Cycle 3 respondents on the index/slice-highlighting item; chosen as most helpful by 5 of 13; and singled out independently by all three peer developers.
Assessment robustness	The single most consistent negative finding. Cycle 1 reported double selections and typed-output items that did not accept an answer; Cycle 2 asked for coverage of negative indexing and edge cases and reported repetitive practice; all three peer developers questioned marking accuracy, answer coverage, or question variety; 3 of 13 Cycle 3 respondents distrusted a mark. Prompted the all-module R5 sweep and the `ChoiceList` fix.
Practice coverage and difficulty	Cycle 2 and Cycle 3 both asked for graded difficulty and fuller per-module coverage: "practice questions are usually easy then comes the exams", and each module "only covers some parts, it doesn't exhaust the whole topic". The lowest-scoring pedagogical claim in Cycle 3 (mean 3.85, 3 disagreements).
Feedback clarity	Mostly rated clear; several Cycle 2 respondents specifically valued that feedback named the misconception rather than only marking the answer wrong. Two Cycle 1 respondents wanted more detail, one found figurative wording ("carved into stone") unclear, and 2 of 13 Cycle 3 respondents selected "feedback was unclear".
Navigation and wayfinding	Cycle 1 found no route back to the module overview from inside a module, and one participant froze the page by searching for an unopened module. By Cycle 3 only 1 of 13 reported navigation as confusing, but 5 of 13 were unsure what to do next at some point, making wayfinding the largest remaining usability issue.
Reliability	Cycle 1: a code-run button that did nothing on one attempt, exam-section bugs, and a duplicate-click bug in a slicing question that plausibly corresponds to the R5 `ChoiceList` defect. Cycle 2: the visualiser continued stepping past a detected syntax error.
Localisation	Mixed and specific. One Cycle 1 participant valued composing code in their first language without an intermediate translation into English; the same cycle flagged some terms as isiZulu rather than isiXhosa and boolean literals left in English after switching. A Cycle 3 respondent reported having to keep checking the keyword mapping. 3 of 13 Cycle 3 respondents still selected "isiXhosa terms unclear or inconsistent".
Presentation	Cycles 1 and 2 both reported dark-mode contrast problems, in the "Common mistake" boxes and in green-on-dark code text. The supervisor separately noted positively the use of colour to distinguish elements.
Progression	The 80% module-completion threshold was not communicated in the interface before a learner is blocked from advancing (Cycle 1), and one Cycle 1 respondent argued the traversal order should not be enforced at all, while a Cycle 2 respondent argued the opposite, that the unlock bypass should not be learner-accessible.
Scope requests	Recurring, out-of-scope requests: more Python topics such as recursion (two Cycle 3 respondents, two Cycle 2), a read-aloud mode for audio learners, hoverable keyword documentation, a discussion space for learner questions, timed exam simulation, and a lecturer analytics dashboard (peer review).

Table S14.1: Formative findings that were addressed or partially addressed, with the evidence for each claim. "Re-test" records what a later cycle observed for the same issue. Findings that remain open are in Table S14.2.

Finding	Raised by	Status	Action and evidence, including re-test
Correct options a learner never clicked were rendered as selected after grading	Cycle 1 (duplicate-click on a slicing item); peer review (marking accuracy)	Addressed	The R5 all-module sweep was run in response to the convergent grading reports and surfaced the defect in `ChoiceList`. The fix distinguishes four post-grading states, so only what a learner clicked renders as selected. Reported in Section 6.
Practice repetitive; duplicate or near-duplicate options; too many similar questions	Cycle 1, Cycle 2, peer review	Partial	The practice core was generalised from Slicing alone to all eight modules, and the cross-module sweep found 0 duplicate options over 1,600 generated questions with 100% answer recoverability (Supplement S11). Question variety within a module remains a live request.
Negative indexing not always handled in practice or grading	Cycle 2, peer review	Addressed (backend)	The 232-program generated tracer corpus explicitly covers negative indices and omitted slice bounds, and negative-index normalisation is part of the tracer AST contract (Supplement S4).
Learners distrust whether an answer was graded correctly	Cycle 2, peer review, Cycle 3 (3/13)	Partial	Tampered-token rejection is 100% and recoverability 100% across all eight modules, and the presentation defect above is fixed. Trust itself was not re-measured after the fix, so this is recorded as partially addressed.
\n must be shown as one character	Supervisor (accepted with comment)	Addressed	Character-cell rendering derives from the tracer's index resolution (Section 4.2). Re-test: a Cycle 3 respondent four days later named the length of a string containing an embedded newline as what helped them most.
Code Space: run button did nothing on one attempt	Cycle 1	Addressed	Re-test: no dead-button report in Cycles 2 or 3; the supervisor accepted Code Space; 1 of 13 Cycle 3 respondents found Code Space confusing, which is a learnability rather than a failure report.
No route back to the module overview; page froze on a locked module	Cycle 1	Addressed (partner's frontend)	Re-test: 1 of 13 in Cycle 3 reported navigation as confusing, and no freeze was reported in Cycles 2 or 3.
Practice needs graded difficulty and fuller per-module coverage	Cycle 2, Cycle 3	Partial	The practice core already carries a tier mechanism and weak-concept weighting (Section 4.4), but difficulty is not surfaced to the learner and per-module template breadth is not yet exhaustive.

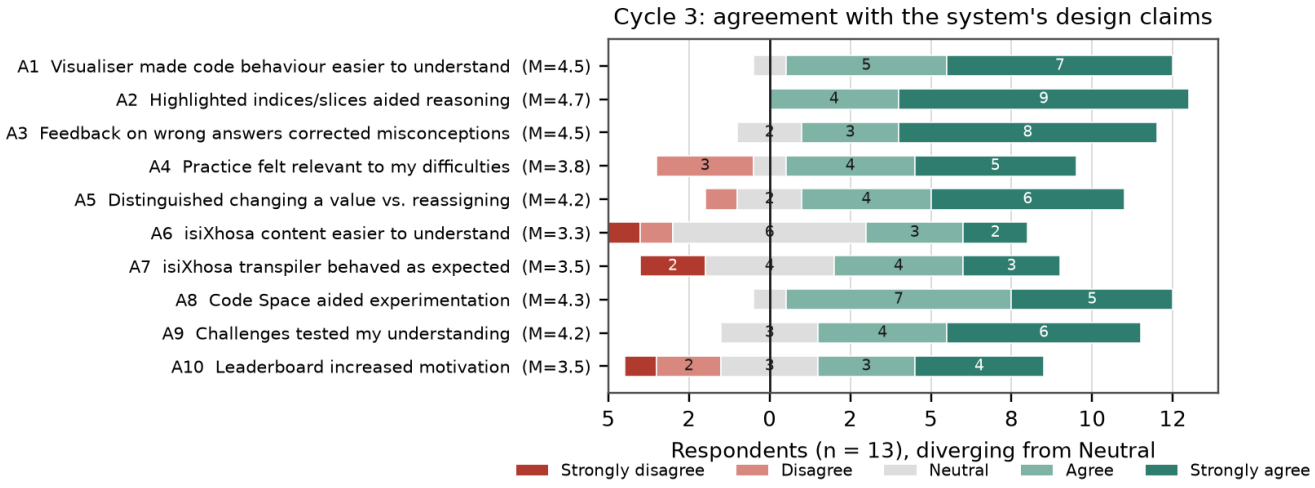


Figure S15.1: Cycle 3 agreement with the system's design claims, one statement per claim, diverging from Neutral ($n = 13$). Segment labels give respondent counts; the mean of each item is shown beside its label. The two visualisation claims (A1, A2) drew no disagreement at all, while the adaptive-practice relevance claim (A4) and the two isiXhosa claims (A6, A7) are the weakest.

Table S14.2: Formative findings that remain open at submission, are the project partner's to resolve, or fall outside the agreed scope. Nothing here is presented as fixed. Findings that were addressed are in Table S14.1.

Finding	Raised by	Status	Position at submission
Wayfinding: unclear what to do next	Cycle 1, Cycle 3 (5/13)	Open	The largest remaining usability finding. One of the five clarified that this was exploration rather than confusion, but four did not.
Some code-writing items display the expected output, which a learner can print verbatim	Peer review	Open	A genuine validation weakness in the challenge corpus. Not fixed at submission; recorded as future work.
isiXhosa: some terms are isiZulu; boolean literals left in English; keyword mapping hard to recall	Cycle 1, Cycle 3 (3/13)	Open	Not resolved. The Cycle 3 checklist rate did not fall relative to the earlier cycles, and the two Cycle 3 isiXhosa agreement items are the lowest of the ten. Translation content is the partner's; the keyword table is shared with the transpiler (Supplement S5).
Visualiser continues stepping past a detected syntax error	Cycle 2	Open	Recorded; not fixed at submission.
Error explanations in Challenges should explain the error codes	Peer review, Cycle 3 free text	Open	isiXhosa runtime-error templates exist (Section 4.3), but Challenge-level error explanation is not implemented.
Code Space should offer an option to clear the code	Supervisor (accepted with comment)	Open	Recorded; not implemented at submission.
80% module-completion threshold not communicated before it blocks progress	Cycle 1	Open	Recorded; not verified as fixed.
Explanation wording too figurative or dense ("carved into stone", the slicing knife analogy)	Cycle 1, Cycle 2	Open (partner's content)	Recorded and passed to the lesson-content contributor.
Dark-mode contrast in "Common mistake" boxes and green-on-dark code text	Cycle 1, Cycle 2	Open (partner's frontend)	Recorded. Not reported again in Cycle 3, but not verified as fixed.
Lecturer analytics dashboard; more Python topics; read-aloud mode; hoverable keyword docs; discussion space; timed exams	Peer review, Cycles 1–3	Out of scope	Recorded as future work. None are within the agreed project scope.

Table S15.1: Cycle 3 design-claim agreement battery ($n = 13$), ordered by mean. Each statement was written to correspond to one specific design claim, so that agreement can be attributed to a component rather than to the system in general. "Agree" counts *agree* plus *strongly agree*; "Disagree" counts *disagree* plus *strongly disagree*. SD/D/N/A/SA gives the full response vector.

Tag	Statement	Mean	Med.	Agree	Neutral	Disagree	SD/D/N/A/SA
A2	Seeing highlighted indices/slices helped me reason about string operations	4.69	5	13	0	0	0/0/0/4/9
A1	The step-by-step visualiser made code behaviour easier to understand	4.46	5	12	1	0	0/0/1/5/7
A3	The feedback after wrong answers helped me correct misconceptions	4.46	5	11	2	0	0/0/2/3/8
A8	The Code Space section helped me experiment and learn effectively	4.31	4	12	1	0	0/0/1/7/5
A9	Challenges were useful for testing my understanding	4.23	4	10	3	0	0/0/3/4/6
A5	The tool helped me distinguish changing a value from reassigning a variable	4.15	4	10	2	1	0/1/2/4/6
A4	Practice questions felt relevant to what I was struggling with	3.85	4	9	1	3	0/3/1/4/5
A10	The leaderboard increased my motivation to practise	3.54	4	7	3	3	1/2/3/3/4
A7	The isiXhosa transpiler worked as I expected	3.46	4	7	4	2	2/0/4/4/3
A6	The isiXhosa translation support made the content easier to understand	3.31	3	5	6	2	1/1/6/3/2

Mean of the five core pedagogical items (A1–A5): 4.32 Mean of the two isiXhosa items (A6, A7): 3.38

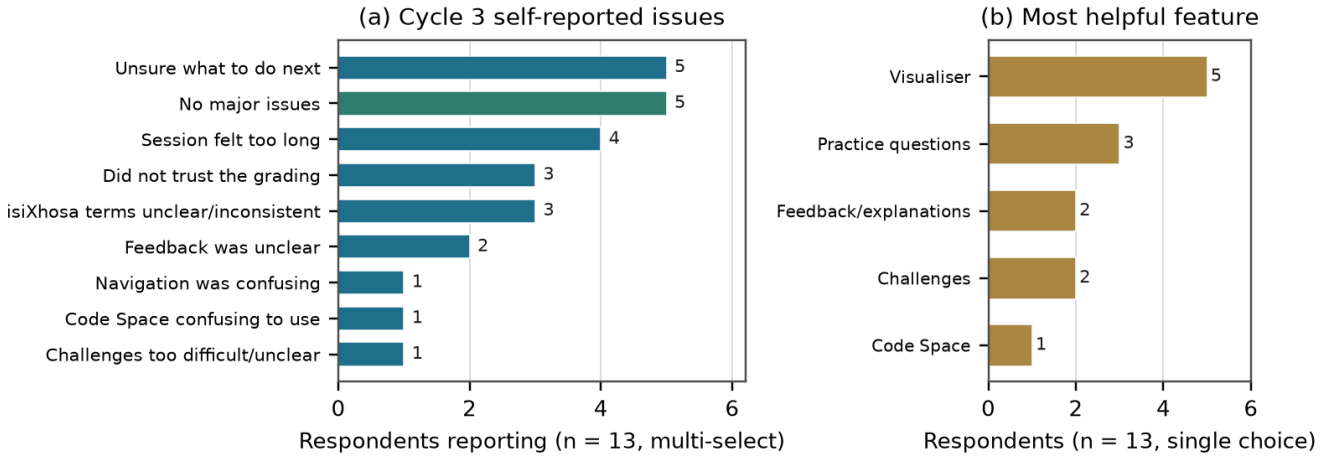


Figure S15.2: Cycle 3 reported problems and most-helpful feature ($n = 13$). Part (a) is the fixed checklist, which participants could multi-select; "No major issues" is shown in a distinct colour because it is not a problem report. Part (b) is a single forced choice, so the counts sum to 13.

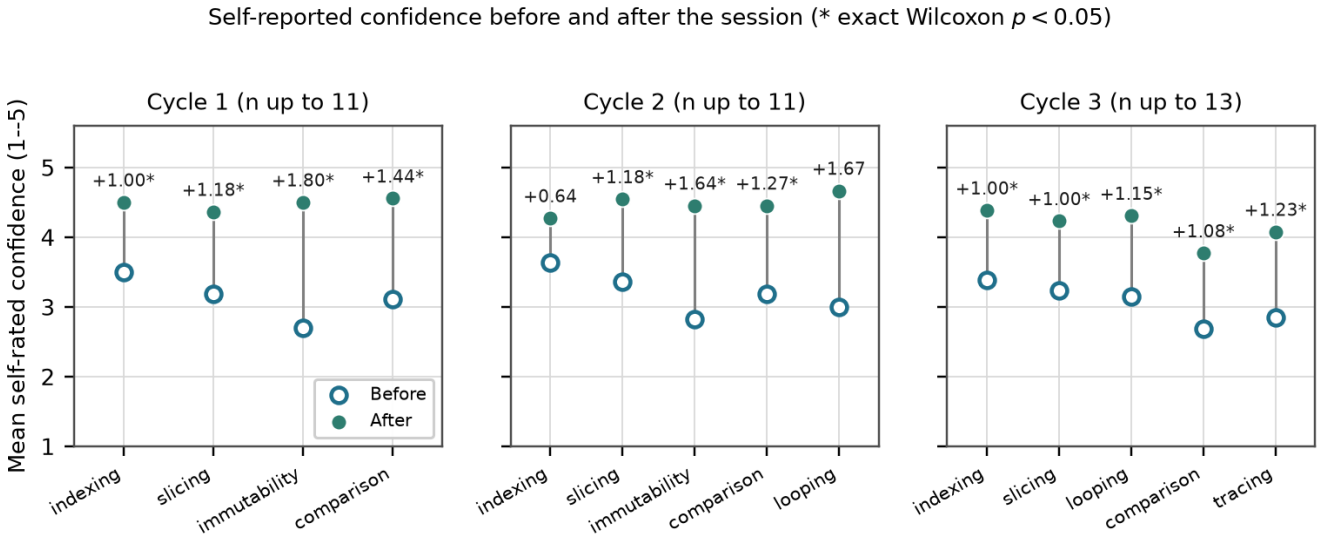


Figure S16.1: Retrospective pre–post self-rated confidence by concept, per cycle. Hollow markers are the "before this session" mean and filled markers the "after this session" mean; the annotation is the mean paired change, starred where the exact Wilcoxon test gives $p < 0.05$. Both ratings were collected after the session on one questionnaire, so these are perceptions of change rather than a measured learning gain.

Table S15.2: Cycle 3 fixed issue checklist ($n = 13$, multi-select). Options were derived from the free-text problems raised in Cycles 1 and 2, so this measures the prevalence of known issues with the target population.

Option selected	Count
I was unsure what to do next at some point	5/13
No major issues	5/13
The session felt too long	4/13
I did not trust whether my answer was graded correctly	3/13
isiXhosa translation terms were unclear or inconsistent	3/13
Feedback was unclear	2/13
Navigation was confusing	1/13
Code Space was confusing to use	1/13
Challenges felt too difficult or unclear	1/13

Table S15.3: Single-item isiXhosa helpfulness rating (1–5) across the three cycles. Blank responses were not imputed.

Cycle	Answered	Mean	Median	Rated 4–5
Cycle 1	7/11	3.00	4.0	4/7
Cycle 2	10/11	3.90	4.5	7/10
Cycle 3	10/13	3.70	4.0	6/10
Pooled	**27/35**	**3.59**	**4.0**	**17/27**

Table S16.1: Retrospective pre–post confidence by concept and cycle. "Up/=/Dn" counts participants whose rating rose, stayed the same, and fell. p is two-sided and exact by full sign-flip enumeration; r is the matched-pairs rank-biserial correlation.

Cycle	Concept	n	Pre	Post	Δ	Up/=/Dn	W^+	p	r
Cycle 1	String indexing	10	3.50	4.50	+1.00	7/3/0	28.0	0.016	+1.00
Cycle 1	String slicing	11	3.18	4.36	+1.18	8/3/0	36.0	0.008	+1.00
Cycle 1	String immutability	10	2.70	4.50	+1.80	8/2/0	36.0	0.008	+1.00
Cycle 1	String comparison	9	3.11	4.56	+1.44	6/3/0	21.0	0.031	+1.00
Cycle 2	String indexing	11	3.64	4.27	+0.64	3/7/1	9.0	0.250	+0.80
Cycle 2	String slicing	11	3.36	4.55	+1.18	6/4/1	26.5	0.047	+0.89
Cycle 2	String immutability	11	2.82	4.45	+1.64	7/2/2	41.0	0.031	+0.82
Cycle 2	String comparison	11	3.18	4.45	+1.27	7/3/1	34.5	0.023	+0.92
Cycle 2	String looping	6	3.00	4.67	+1.67	5/0/1	19.5	0.094	+0.86
Cycle 3	String indexing	13	3.38	4.38	+1.00	8/5/0	36.0	0.008	+1.00
Cycle 3	String slicing	13	3.23	4.23	+1.00	8/5/0	36.0	0.008	+1.00
Cycle 3	String looping	13	3.15	4.31	+1.15	8/5/0	36.0	0.008	+1.00
Cycle 3	String comparison	13	2.69	3.77	+1.08	7/6/0	28.0	0.016	+1.00
Cycle 3	Tracing code step-by-step	13	2.85	4.08	+1.23	10/3/0	55.0	0.002	+1.00
Participant-level mean confidence over all concepts in the cycle									
Cycle 1	all concepts	11	3.16	4.50	+1.34	9/2/0	N/A	0.004	+1.00
Cycle 2	all concepts	11	3.28	4.47	+1.19	8/2/1	N/A	0.008	+0.96
Cycle 3	all concepts	13	3.06	4.15	+1.09	10/3/0	N/A	0.002	+1.00
Pooled	all concepts	**35**	**3.16**	**4.36**	**+1.20**	**27/7/1**	N/A	**< 0.001**	**+0.99**

Table S17.1: Cross-team peer review by three developers of a comparable Python-learning tool. Reported verbatim points are condensed; the full forms are retained with the project records.

Rev.	Endorsed	Questioned or reported broken	Overall
R1	Worked through modules, practice, visualising, and the exam. Lessons "thorough & informing". String visualisation "very interactive & intuitive to use"; "unique & fun". Leaderboard implementation.	An exam item asked for the index of `"a"` in `"xhosa"`; the reviewer answered -1 and was marked wrong. The expected answer is 4, so the mark was defensible, but the item's wording did not signal whether a search-miss convention applied.	"The system was well-designed. I liked the leaderboard implementation & visualisation feature."
R2	Tried modules first; the string visualisation was "really cool". Site "very clean, everything worked nicely"; concepts "explained and tested in fun ways".	"The marking is not 100% accurate." Some items did not include the expected answer among the options. Some items displayed the expected output, so a learner could satisfy them by printing it verbatim. Negative indexing not always taken into account.	"I enjoyed how you made strings in Python enjoyable to learn." Notes that PyTutor has a lecturer dashboard surfacing what students struggle with and ask, offered as something worth knowing.
R3	"Very clean look, works well & explains well." Module structure, the visualiser, and the leaderboard concept.	Exam answers lacked full path coverage, with cases where a wrong answer could be accepted; questions should be more unique, as there were too many similar ones. Error handling in Challenges should explain the error codes.	"Very helpful; impressed by the visualiser and the sandbox environment. Gives good explanations to problems/modules."

Table S17.2: Supervisor acceptance walkthrough of the deployed production build, 10 September 2026. A = accepted; no item was marked as requiring a fix.

Category	Items	A	Comments recorded
Account & access	5	4	"All good" on guest-mode scoping; email-link sign-up left blank
Lesson modules (all 8, each covering content, worked example, visualiser, warm-up, practice)	8	8	"Good visuals. Easy to follow & navigate."
Core tools (Code Space, standalone visualiser, step-by-step tracer)	3	3	Code Space: add an option to clear the code
Practice & grading (MCQ, select-all, short answer, code writing, misconception feedback)	5	5	N/A
isiXhosa localisation (content toggle, graded feedback, UI strings)	3	3	N/A
Other features (Challenges, leaderboard, settings)	3	3	"Nice additions."
Site-wide (navigation without dead ends, production URL, no broken links or console errors)	3	3	"Good!"
Total	**30**	**29**	**2 accepted with a corrective comment; 0 requiring a fix**

Outstanding item recorded in the overall assessment: the newline escape \n must be shown as one character.

Part E Full-Size System Diagrams

Placed last because each diagram occupies a full page; interleaving them with the tables above would fragment both.

S18 System Diagrams

This supplement collects the diagrams that support Sections 3 to 4 without being needed to follow them. Figure S18.1 summarises the shared adaptive-practice loop; Figure S18.2 details the tracer's operation; Figure S18.3 shows measured sandbox containment; Figure S18.4 expands practice selection, grading, feedback, and tier progression into a detailed swimlane activity diagram; Figure S18.5 expands the compact three-tier overview in Figure 1 into a detailed deployment and dependency view; Figure S18.6 gives the tracer's internal call sequence; Figure S18.7 is a UML sequence diagram of one execution request; and Figure S18.8 gives the PostgreSQL schema.

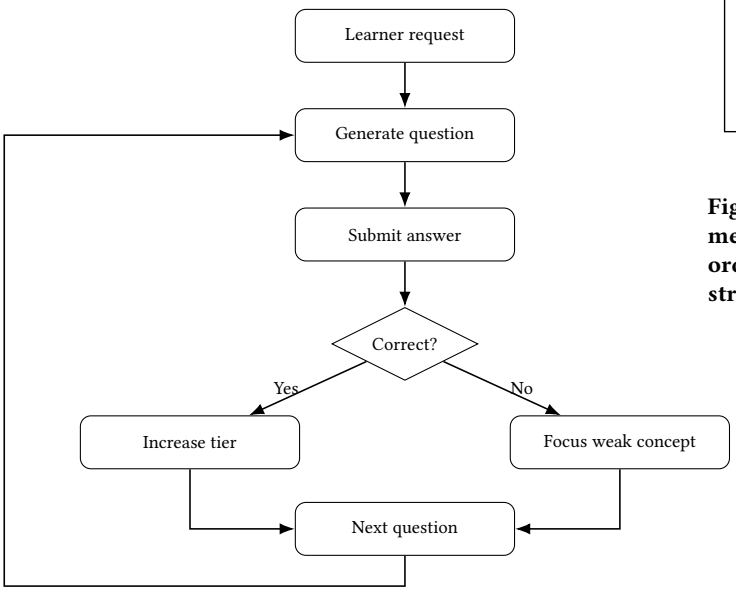


Figure S18.1: Adaptive practice workflow. The Slicing module illustrates the shared generation, grading, and progression process used across all eight practice modules.

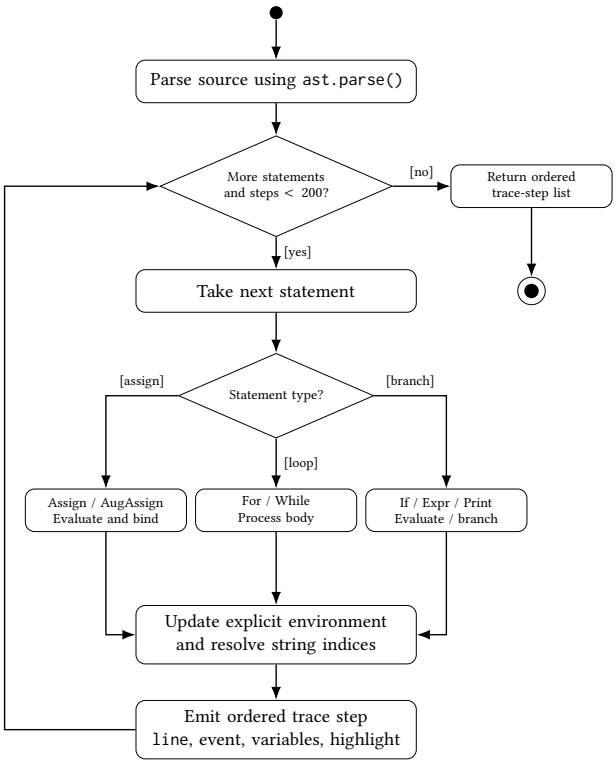


Figure S18.2: AST-based execution tracer. Supported statements update an explicit environment and are converted into ordered trace steps containing program state and resolved string indices.

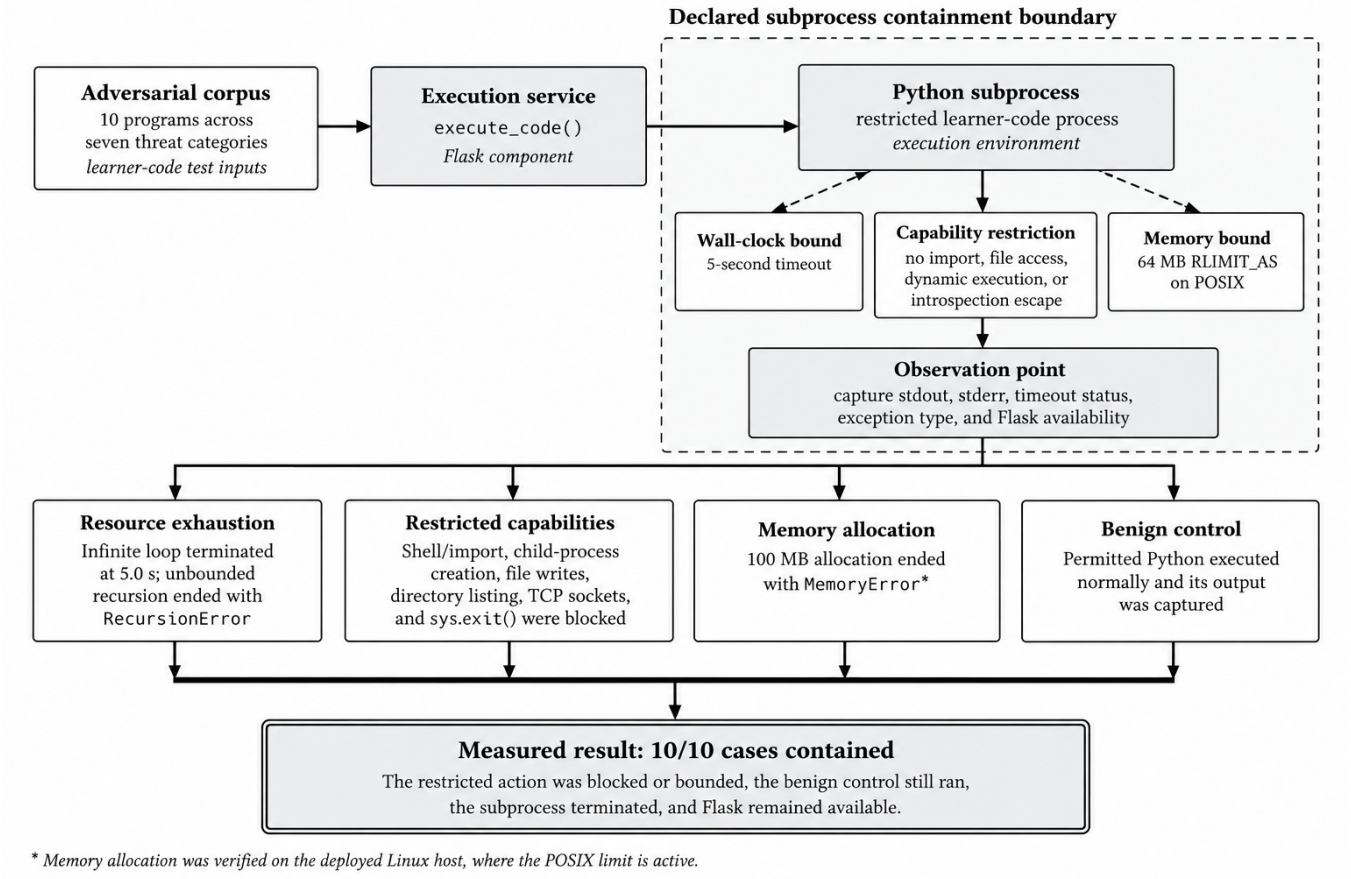


Figure S18.3: UML deployment-and-activity view of the measured sandbox containment. The ten pre-declared cases were submitted through the same execution service as learner code. The subprocess combines a five-second wall-clock limit, restricted builtins, and a POSIX address-space limit; the observation point records termination, exceptions, output, and continued Flask availability. All ten cases were contained within the seven declared threat categories, while the benign control still executed. This is evidence for the stated threat set, not a general security guarantee.

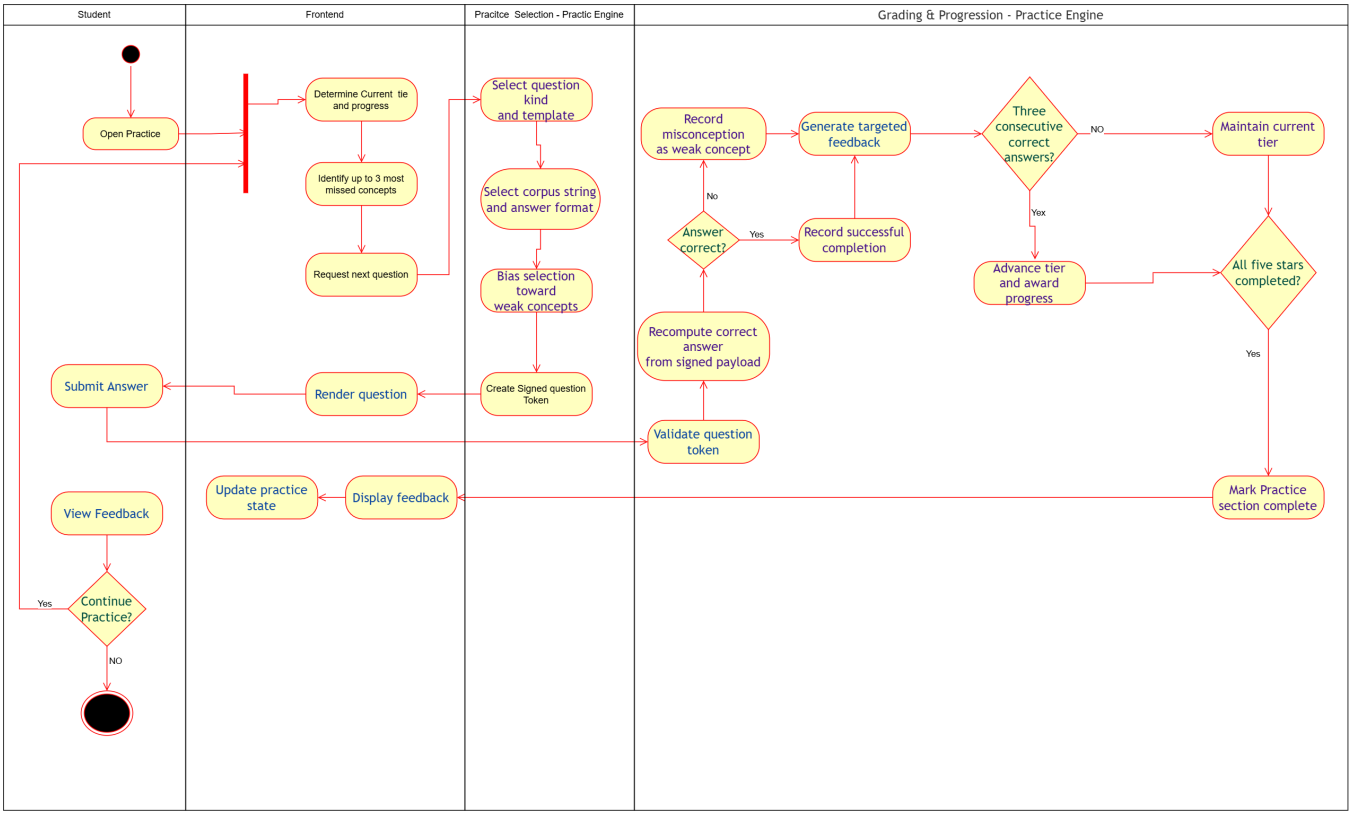


Figure S18.4: Detailed swimlane activity diagram for the adaptive-practice engine. The lanes separate learner interaction, frontend state, question selection, and backend grading and progression. The flow shows weak-concept weighting, procedural question composition, signed-token validation, answer recomputation, misconception recording, targeted feedback, tier advancement, star awards, and section completion. This expands the compact workflow in Figure S18.1.

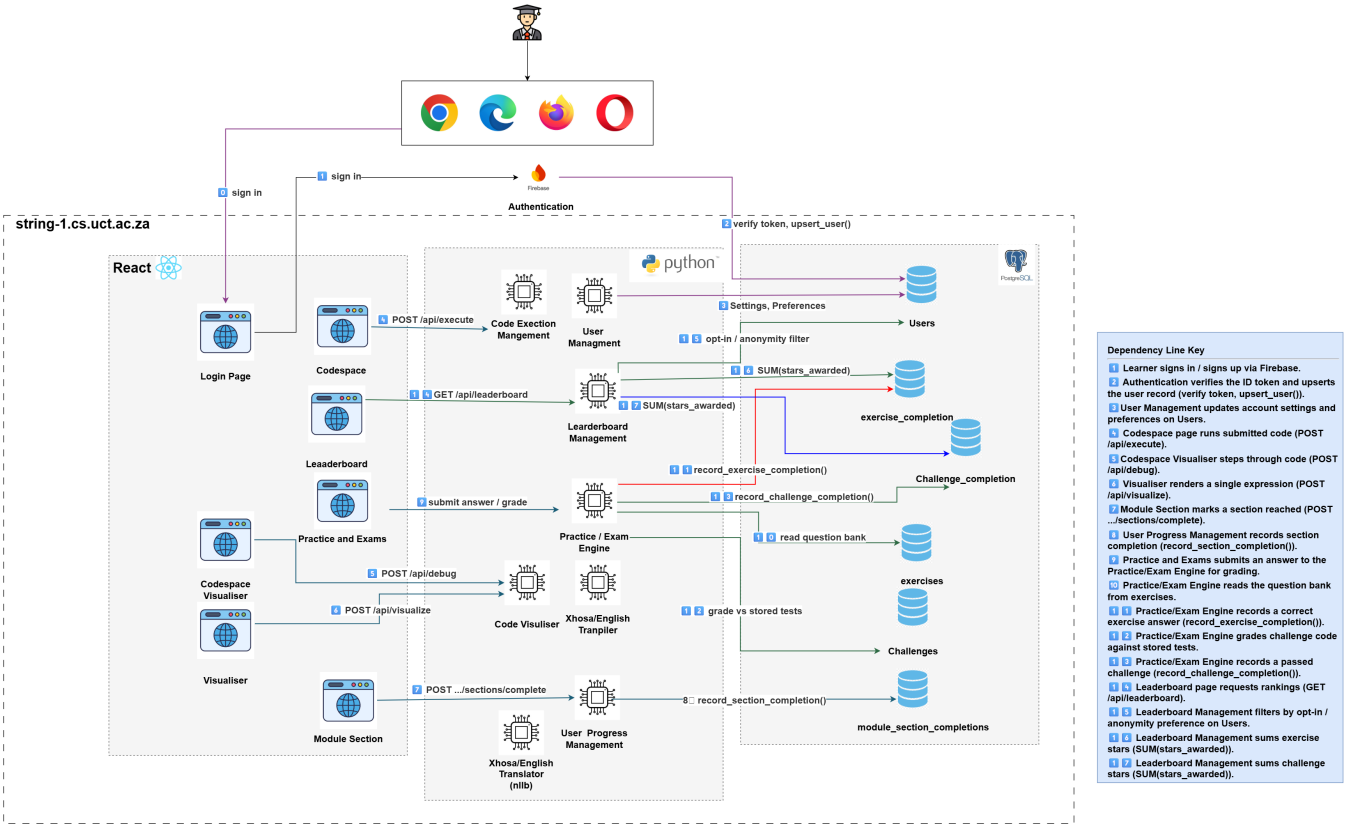


Figure S18.5: Detailed deployed architecture of String Conundrum, expanding the compact three-tier overview in Figure 1. It shows the learner-facing React views, Firebase authentication, backend services, PostgreSQL relations, principal API paths, and numbered dependencies. The key on the right explains the numbered flows; the realised execution-request sequence is specified separately in Figure S18.7.

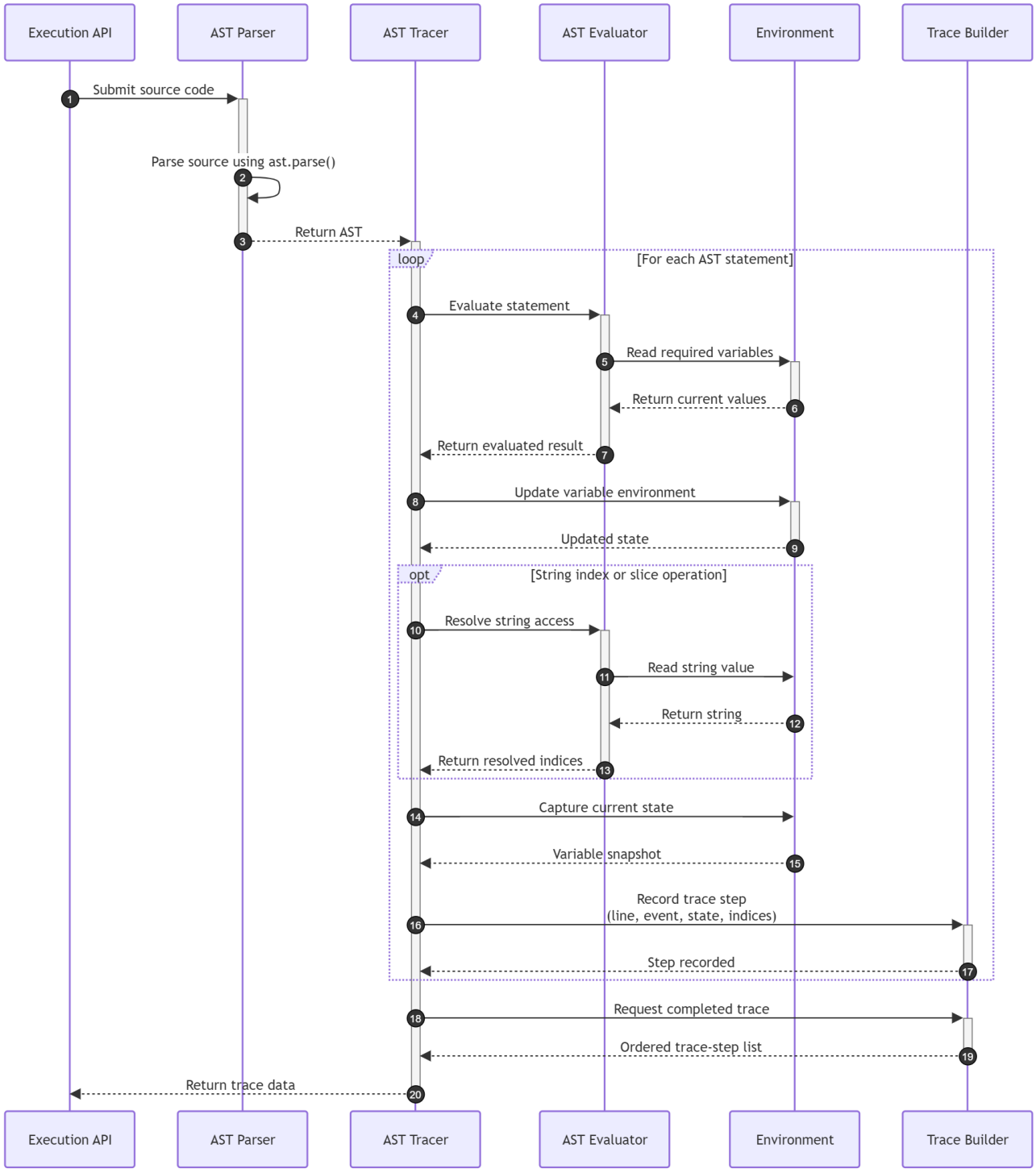


Figure S18.6: Sequence diagram of one `trace_steps` call. The AST is parsed once. Then each statement is evaluated against the environment dictionary in a loop, with string index and slice resolution as an optional sub-sequence. Every step is then captured and handed to the ordered trace-step list returned to the caller.

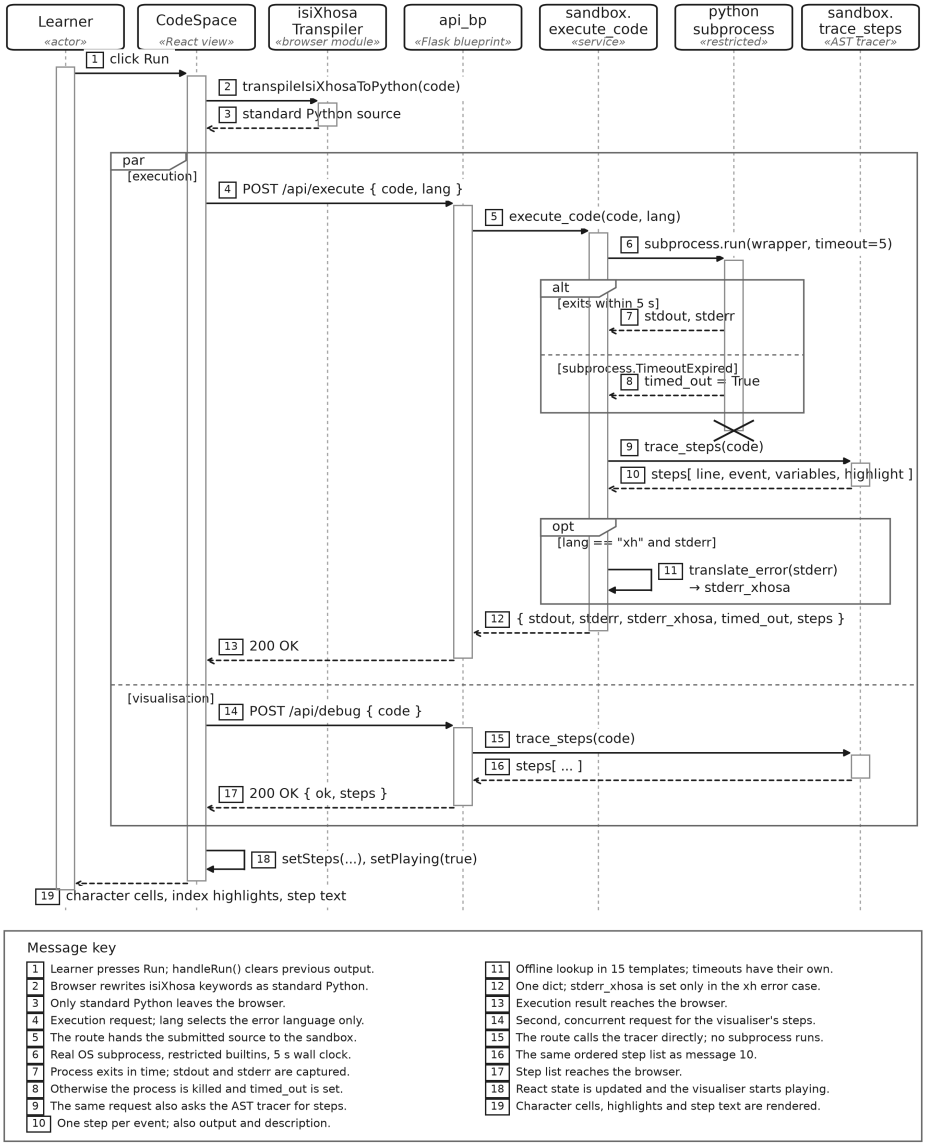


Figure S18.7: UML sequence diagram of one execution request, from the learner pressing Run to the rendered visualisation; numbered badges are explained in the Message key panel at the foot of the figure. Four points matter for the backend contribution. Optional isiXhosa transpilation (messages 2–3) completes in the browser, so the sandbox and tracer only ever receive standard Python. The `par` fragment is `handleRun()` issuing `/api/execute` and `/api/debug` together through `Promise.all`, which is why the tracer appears on both request paths. The `alt` fragment separates a subprocess that exits inside its five-second budget from one killed by `subprocess.TimeoutExpired`, which sets `timed_out` and writes "Execution timed out (5 second limit)." to `stderr`. The `opt` fragment is the offline isiXhosa error-template lookup (Section 4.3), attempted only when the request carries `lang=xh` and execution failed. The restricted-builtins environment and 64 MB `RLIMIT_AS` cap behind message 6 are described in Section 4.1, and the cross on the subprocess lifeline is a destruction marker: the process does not outlive the call.

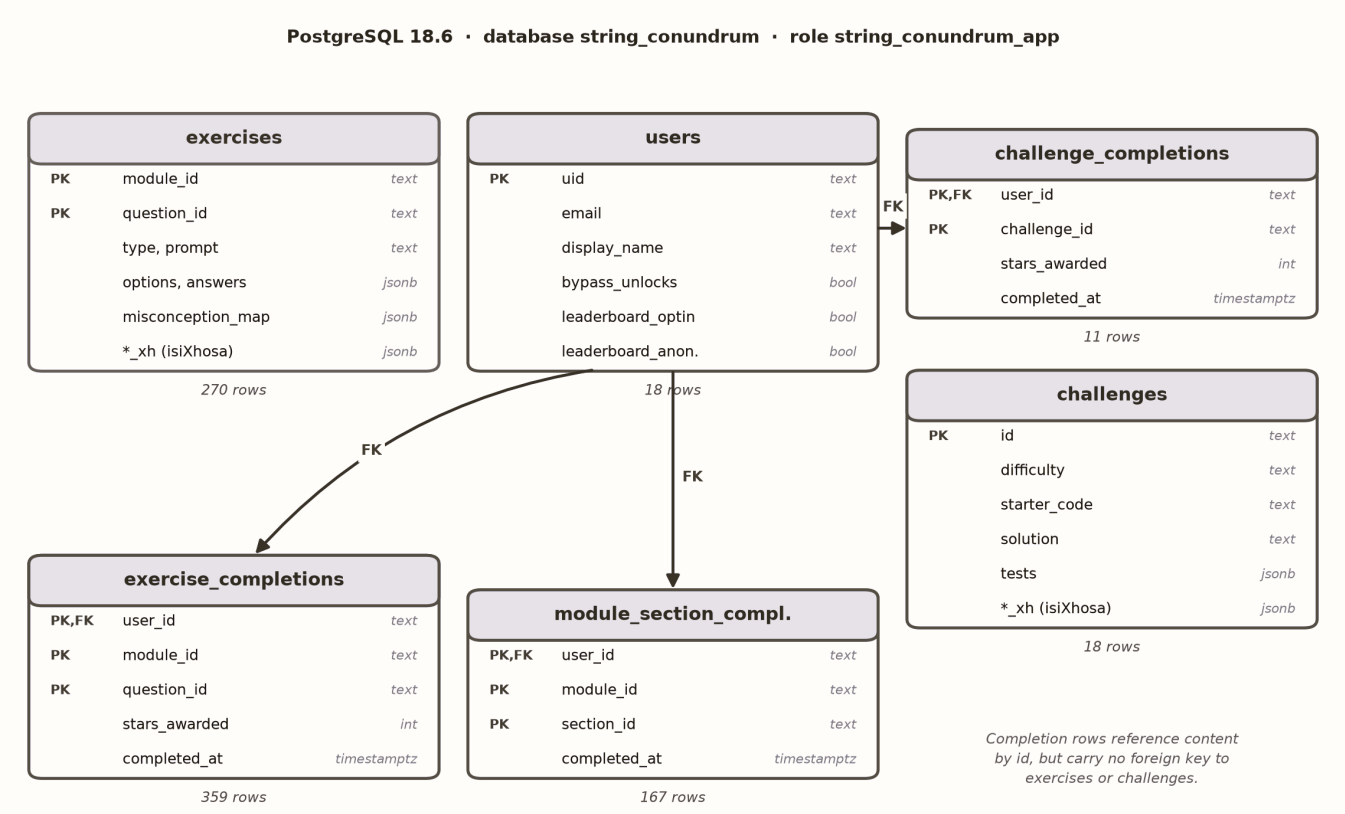


Figure S18.8: Full-size PostgreSQL schema diagram for the persistence layer described in Section 4.5.

Figure S18.9 is included for context only. The misconception-feedback system it diagrams is a project partner's contribution, not the author's. It is shown here because the author's progress-recording and authentication components sit on either side of it, and because the adaptive practice engine described in Section 4.4 calls into it for shared feedback text.

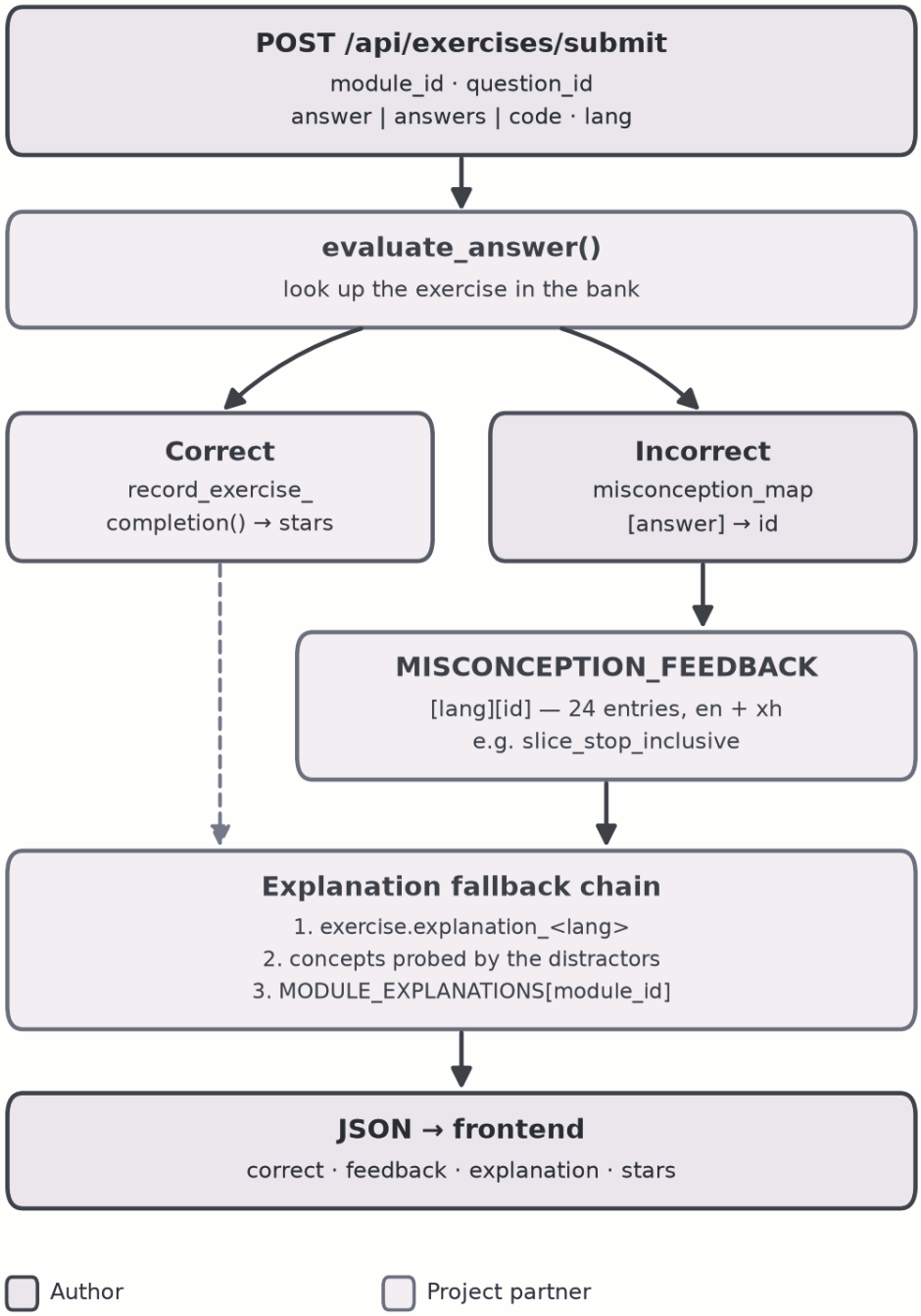


Figure S18.9: The misconception-feedback path through `/api/exercises/submit`. Shaded boxes are the project partner's components. The entry and exit points are the author's.